Welcome Banner

Here's the data flow for the Welcome Banner, end to end.

The four pieces in play

1. **Salesforce data source** — the running user's record in the User object, accessed via Lightning Data Service (no Apex).
2. **@wire(getRecord, ...)** — the adapter that fetches that single record.
3. **Getters** — userFullName (derived from the wired record) and todayDate (purely client-side, no Salesforce data).
4. **Template** — reads {userFullName} and {todayDate} and renders the banner.

Notice there is **no Apex class** here. This is the deliberate contrast with the Case Dashboard: when all you need is fields from a single, known record, LDS does it natively. No controller to write, no test class to maintain, automatic FLS, and the record stays in sync with any other component reading the same data on the page.

Step 1 — Imports set up the wire's "address"

```
import { getRecord } from 'lightning/uiRecordApi';

import userId from '@salesforce/user/Id';

import USER_FIRSTNAME from '@salesforce/schema/User.FirstName';

import USER_LASTNAME from '@salesforce/schema/User.LastName';

const FIELDS = [USER_FIRSTNAME, USER_LASTNAME];
```

Each import answers a different question:

- getRecord — **how** to fetch (the wire adapter from UI API).
- userId from @salesforce/user/Id — **whose** record to fetch. This is a special scoped module that gives you the running user's Id at runtime; no need to call Apex or UserInfo. The same family includes @salesforce/user/isGuest, @salesforce/user/Name, etc.
- USER_FIRSTNAME / USER_LASTNAME from @salesforce/schema/User.FieldName — **which** fields. These aren't strings; they're typed schema references. The advantage is the platform tracks the dependency: if anyone tries to delete User.FirstName, Salesforce blocks the change because this component references it. They also fail at compile time if the API name is wrong.

FIELDS is just an array of those references that the wire adapter expects.

Step 2 — Component is constructed

When the page loads the framework instantiates WelcomeBanner. There are no @track fields here — the only state is the wired user property (an object the wire will populate) and the two getters.

At this initial instant, this.user is undefined, so:

- userFullName getter returns '' (the guard if (this.user && this.user.data) is false).
- todayDate getter returns today's date — it doesn't depend on any Salesforce data, so it works immediately.

The template renders, the "Welcome back" label and date appear, and the name slot is briefly blank.

Step 3 — @wire fires automatically

@wire(getRecord, { recordId: userId, fields: FIELDS })

user;

Read this as: *"bind the user property to the live result of getRecord({ recordId, fields })."* The framework calls the adapter with that config the moment the component is ready.

Behind the scenes the UI API sends a request like *"fetch User record 005xxx... and return FirstName, LastName"*. LDS first checks its client-side cache — if any other component on the page has already read this same User with these same fields, the answer is returned instantly without a server round-trip. Otherwise, one HTTP call goes to Salesforce.

A subtle but important point: recordId: userId (no $ prefix) means the wire is using a *static* value resolved once at component creation. That's correct here because the running user's Id doesn't change while the page is open. If recordId were a property that might change ('$recordId'), the wire would re-fire automatically whenever the property changed.

Step 4 — The wire delivers the result

The wire writes its result into this.user. Unlike the Case Dashboard where the callback shape was { data, error }, here the wire is bound to a **property** so the *property itself* takes on that shape:

```
this.user = {
  data: {
    id: '005xxx...',
    apiName: 'User',
    fields: {
      FirstName: { value: 'Ananya', displayValue: null },
      LastName:  { value: 'Kumar',  displayValue: null }
```

```
    }
  },
  error: undefined
}
```

Note the nesting: it's user.data.fields.FieldName.value. The wrapper around each field exists because the UI API returns more than just the raw value — displayValue carries the formatted version (currency symbols, date formatting) when applicable, which is why dates and currencies come back as both a raw and a display string.

Because this.user was assigned a new reference, the framework marks the component dirty and schedules a re-render.

Side note on @track: you might notice there's no @track user; here, yet the UI updates. That's because modern LWC (API ≥ 56 in particular) makes wire-bound properties reactive automatically. @track is only needed when you mutate nested fields of an object/array in place — which never happens to a wired result, since the wire always assigns a fresh reference.

Step 5 — userFullName getter computes the display string

```
get userFullName() {
  if (this.user && this.user.data) {
    const first = this.user.data.fields.FirstName.value || '';
    const last  = this.user.data.fields.LastName.value  || '';
    return (first + ' ' + last).trim();
  }
  return '';
}
```

A getter is just a function the template calls every time it needs the value. It's not stored, it's not cached — each access recomputes it. So this runs once now that this.user.data exists, walks down to the two field values, and returns "Ananya Kumar".

A few defensive details worth pointing out:

- this.user && this.user.data — guards against the brief window where the wire hasn't returned yet, and also against the error case (where data is undefined and error is populated).
- || '' — if either field is missing on the User record, fall back to an empty string instead of letting undefined slip into the concatenation.

- .trim() — handles users with only a first name ("Ananya " → "Ananya").

Step 6 — todayDate runs entirely client-side

```
get todayDate() {
   return new Date().toLocaleDateString('en-IN', {
      weekday: 'long', day: 'numeric', month: 'long', year: 'numeric'
   });
}
```

Nothing here touches Salesforce. new Date() reads the browser's clock, and toLocaleDateString('en-IN', ...) formats it for the India locale. With those four options it produces something like *"Sunday, 15 June 2026"*.

This is invoked once per render. Since the component doesn't re-render after the initial wire callback (no further state changes), the date is essentially computed twice — once on the initial render, once after the wire — and never again. If the user kept the page open past midnight, the date would not refresh; you'd need a setInterval in connectedCallback to update it.

Step 7 — Template renders the final UI

```
<p ...>{userFullName}</p>   <!-- "Ananya Kumar" -->
<p ...>{todayDate}</p>      <!-- "Sunday, 15 June 2026" -->
```

The framework substitutes the getter values into the markup and the banner shows the name and today's date.

The flow in one picture

Page loads

Component constructed

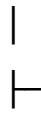
user = undefined

userFullName getter → " (guarded)

todayDate getter → 'Sunday, 15 June 2026'

Template first render — banner shows date, name blank

@wire(getRecord, {recordId:userId, fields:[FirstName, LastName]})



├── checks LDS cache

├── on miss → UI API request to Salesforce

└── populates this.user = { data: { fields: {…} }, error: undefined }

Property change detected → re-render

userFullName getter reruns

→ reads this.user.data.fields.FirstName.value ('Ananya')

→ reads this.user.data.fields.LastName.value ('Kumar')

→ returns 'Ananya Kumar'

Template re-renders — banner now shows full name + date

How this compares to the Case Dashboard

It's worth seeing both side by side, since they illustrate the two main read patterns:

Aspect	Case Dashboard	Welcome Banner
Data source	Aggregate counts across many records	A few fields on one known record
Mechanism	Apex @AuraEnabled(cacheable=true) method + 5 SOQL queries	LDS getRecord wire adapter
Code on the server	A controller class to write/test	None

Aspect	Case Dashboard	Welcome Banner
Result shape	Map<String,Object> → JSON {key: count, …}	{ data: { fields: {Name: {value}} }, error }
Where state lives	Multiple @track properties, set in the callback	A single wire-bound user property
FLS/sharing	Bypassed by without sharing for org-wide counts	Enforced automatically by LDS
Caching	Apex method-level caching	Per-record/field caching shared across components

Two flavours of the wire decorator also show through:

- **Function form** (Case Dashboard): @wire(adapter) callback({data, error}) { … } — you control what happens with the result and you can assign to several properties.
- **Property form** (Welcome Banner): @wire(adapter) propertyName; — the property itself takes the {data, error} shape and you read it from getters.

The function form makes sense when you need to transform or split the result; the property form is cleaner when the data feeds straight into the template through getters, which is exactly the case here.

Case Dashboard:

Here's how the data flows end-to-end in your Case Overview Dashboard, from when the doctor opens the page to when the numbers appear on screen.

The five layers in play

Before tracing the flow, here are the parts:

1. **Apex controller** (CaseDashboardController.getCaseCounts) — runs on the Salesforce server, queries the database, returns a Map.
2. **@wire adapter** in the LWC — the bridge that calls the Apex method and pipes the result back.
3. **Reactive properties** (@track totalOpenCases, etc.) — hold the numbers in the component's state.
4. **Getters** (totalCases, resolutionRate, activeCasesBarStyle, etc.) — derived values computed on the fly from the properties.
5. **Template** — reads properties and getters with {name} and re-renders whenever anything changes.

Now the trace.

Step 1 — Page loads, component is constructed

When the doctor lands on the page, the framework creates an instance of CaseDashboard. The class field initializers run:

@track totalOpenCases = 0;

@track totalResolvedCases = 0;

// ...

@track isLoading = true;

@track hasError = false;

Everything starts at 0, and importantly isLoading = true. So at this instant the template enters the <template if:true={isLoading}> branch — the spinner shows, no stats render.

Step 2 — @wire fires automatically

Right after the instance exists, the framework sees this decorator:

@wire(getCaseCounts)

wiredData({ data, error }) { ... }

Read it as: *"call getCaseCounts on the server, and when the result arrives, invoke wiredData with it."* No parameters this time, so the wire fires once and won't re-fire (no $-prefixed reactive params).

Behind the scenes, LWC issues an Aura/Lightning request to the server with the method's qualified name (CaseDashboardController.getCaseCounts). Because the method is annotated @AuraEnabled(cacheable=true), the response will also be cached on the client — a second component asking for the same data on the same page gets it instantly without a server round-trip.

Step 3 — The Apex method runs on the server

Inside getCaseCounts():

```
User currentUser = [
    SELECT Id, Name, ContactId
    FROM User
    WHERE Id = :UserInfo.getUserId()
    LIMIT 1
];
```

UserInfo.getUserId() returns the running user — the doctor who opened the page. The query grabs their Name and ContactId (community/portal users in Salesforce are linked to a Contact record, which is how you tie a User to a Doctor Contact in your data model).

Then five aggregate queries fire — each is a SELECT COUNT() that returns just a number, not records, which is cheap:

```
result.put('totalOpenCases',  [SELECT COUNT() FROM Case WHERE IsClosed = false AND RecordType.Id = '012dM00000D4tODQAZ']);

result.put('totalResolvedCases', [SELECT COUNT() FROM Case WHERE Status = 'Closed' AND RecordType.Id = '012dM00000D4tODQAZ']);

result.put('myActiveCases',    [SELECT COUNT() FROM Case WHERE IsClosed = false AND Assigned_Doctor__r.Name = :currentUser.Name AND ...]);

result.put('myResolvedCases',  [SELECT COUNT() FROM Case WHERE Status = 'Closed' AND Assigned_Doctor__r.Name = :currentUser.Name AND ...]);

result.put('myPrescriptions',  [SELECT COUNT() FROM Prescription__c WHERE Doctor__c = :currentUser.ContactId]);
```

Two things to notice:

- IsClosed = false filters open cases; Status = 'Closed' filters resolved ones. The hard-coded RecordType Id 012dM00000D4tODQAZ restricts to your Clinical Case record type — that's why a doctor's dashboard never counts Support Cases.
- For "my" cases, the filter Assigned_Doctor__r.Name = :currentUser.Name is the child-to-parent dot notation you've seen in your notes — walking from Case up to its Assigned_Doctor lookup and reading the doctor's Name.

For prescriptions, the link is via Doctor__c = :currentUser.ContactId, because your Prescription's Doctor__c field points to a Contact, and the running User's ContactId tells you which Contact represents them.

Finally:

result.put('doctorName', currentUser.Name);

return result;

The whole Map is serialized to JSON and sent back to the browser. So the network payload is roughly:

{ "totalOpenCases": 12, "totalResolvedCases": 47, "myActiveCases": 5,

"myResolvedCases": 18, "myPrescriptions": 23, "doctorName": "Ananya Kumar" }

Step 4 — The result enters the component

The framework wakes up the wiredData function with this object placed inside a wrapper { data, error }:

```
wiredData({ data, error }) {
    if (data) {
        this.totalOpenCases    = data.totalOpenCases;
        this.totalResolvedCases = data.totalResolvedCases;
        this.myActiveCases     = data.myActiveCases;
        this.myResolvedCases   = data.myResolvedCases;
        this.myPrescriptions   = data.myPrescriptions;
        this.doctorName        = data.doctorName;
        this.isLoading = false;
        this.hasError  = false;
    } else if (error) {
        console.error('Error:', error);
        this.hasError = true;
```

```
        this.isLoading = false;
    }
}
```

Each line assigns a property. Because every property is @track, those assignments are reactive — the framework marks the component dirty for a re-render. isLoading flips to false, which means the spinner branch in the template stops matching and the main content branch takes over.

The wrapped { data, error } is the standard shape every wire delivers: exactly one of them is set on each invocation. The console.error is a useful safety net during development; in production you'd typically surface the message in the UI.

Step 5 — Getters compute derived values

The reactive properties only hold raw counts. Everything else is computed lazily when the template reads it:

```
get totalCases() { return this.myActiveCases + this.myResolvedCases; }


get resolutionRate() {
    if (this.totalCases === 0) return 0;
    return Math.round((this.myResolvedCases / this.totalCases) * 100);
}
```

These two feed the circular indicator in the top-right of the summary section.

The bar widths work similarly. First, find the largest of the four counts (with a floor of 1 so we never divide by zero):

```
get maxCases() {
    return Math.max(this.myActiveCases, this.totalOpenCases,
                    this.totalResolvedCases, this.myPrescriptions, 1);
}
```

Then each metric is expressed as a percentage of that maximum, capped at 100%:

```
get myActiveCasesPercent() {
    return Math.min(Math.round((this.myActiveCases / this.maxCases) * 100), 100);
}
```

So if a doctor has myActiveCases = 5 and totalOpenCases = 12, the max is 12, and the active bar comes out to ≈42% of the track width — visually you can see at a glance how your active load compares to the largest metric on the dashboard.

The style getters wrap those percentages in a CSS string the template can drop straight into style={...}:

```
get activeCasesBarStyle() {
   return `height:100%; width:${this.myActiveCasesPercent}%;
        background:linear-gradient(90deg,#2d6a4f,#40916c);
        border-radius:8px; transition:width 0.6s ease;`;
}
```

The transition:width 0.6s ease is what makes the bars animate from 0 to their value when the data lands — purely a CSS effect, triggered automatically by the width changing.

The circular indicator uses a slightly different trick — SVG dash-offset:

```
get resolutionDashOffset() {
   // circumference = 2 * pi * 50 = 314
   return Math.round(314 - (314 * this.resolutionRate / 100));
}
```

The SVG ring has circumference 314 (it's a circle of radius 50). Drawing only part of that perimeter is done by setting stroke-dasharray="314" and a stroke-dashoffset that hides the part you don't want. At 0% resolution, offset = 314 (entire ring hidden); at 100%, offset = 0 (full ring shown); at 60%, offset ≈ 126.

Step 6 — The template renders

Now the template is re-evaluated with isLoading = false and hasError = false, so it enters the main content branch and reads:

```
<p ...>{myActiveCases}</p>        <!-- the property -->
<p ...>{resolutionRate}%</p>      <!-- a getter   -->
<div style={activeCasesBarStyle}>  <!-- a style getter -->
<circle ... stroke-dashoffset={resolutionDashOffset} />  <!-- another getter -->
```

The numbers appear in the four stat tiles, the bars slide out to their widths, the circular ring fills to the resolution percentage, and the header banner shows "Welcome back, **Ananya Kumar**".

The flow in one picture

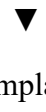
User opens page

LWC constructor — properties default to 0, isLoading=true

@wire fires automatically ──► Apex CaseDashboardController.getCaseCounts()

wiredData({data, error}) callback

Framework detects @track changes → re-render



Template reads:

Bars animate, ring fills, numbers appear

A few things worth knowing

The cacheable=true annotation means LDS will hold the result client-side. If you ever update a case while viewing this dashboard, the numbers won't refresh automatically — you'd need to capture the wire result in a property and call refreshApex(this.wiredResult) after the update, or navigate away and back.

The hard-coded RecordType Id (012dM00000D4tODQAZ) is brittle — Ids change when you deploy to a new org. For interview safety, you'd query it dynamically:

```
Id clinicalRT = Schema.SObjectType.Case.getRecordTypeInfosByDeveloperName()
                .get('Clinical_Case').getRecordTypeId();
```

The match Assigned_Doctor__r.Name = :currentUser.Name works but is fragile — two doctors with the same name would collide. The robust comparison is by Id: Assigned_Doctor__c = :currentUser.ContactId (assuming Assigned_Doctor__c is a lookup to Contact, like Doctor__c on Prescription). Worth mentioning if asked "what would you improve."

And without sharing on the controller means org-wide counts (totalOpenCases, totalResolvedCases) ignore the doctor's record-level access — which is what you want for an "All Open Cases" tile, but it's a deliberate design choice the evaluator may probe.

Quick Links

Here's the data flow for the Quick Links component, end to end.

The four pieces in play

1. **Template** — three identical-looking cards, each with an onclick wired to a different handler method.
2. **NavigationMixin** — a Salesforce-provided mixin that adds the Navigate capability to the component.
3. **Three handler methods** — handleHealthWorker, handleQuickGuide, handleCampaign. Each fires a navigation event.
4. **Salesforce Navigation Service** — the platform-level router that listens for navigation events and actually changes where the user is sent.

Notice what's *not* here: no @wire, no Apex, no @track properties, no LDS. This component holds no state of its own and never talks to the database. It is purely a user-action router — the data flow is entirely *outbound*: a click leaves the component, travels through the Navigation Service, and the user ends up on a new URL. There is no data flowing *in*.

Step 1 — The mixin extends the class hierarchy

import { NavigationMixin } from 'lightning/navigation';

export default class QuickLinks extends NavigationMixin(LightningElement) { ... }

A *mixin* in JavaScript is a function that takes a class and returns a new class with extra capabilities bolted on. NavigationMixin(LightningElement) produces a class that *is* LightningElement plus a Navigate method and a GenerateUrl method.

After this line, the class chain looks like:

QuickLinks —► (NavigationMixin-enhanced LightningElement) —► LightningElement

So this inside QuickLinks is a LightningElement with two extra things added: this[NavigationMixin.Navigate](...) and this[NavigationMixin.GenerateUrl](...). The bracket-with-symbol syntax (this[NavigationMixin.Navigate]) is used instead of dot notation because NavigationMixin.Navigate is a Symbol, not a string — Salesforce uses symbols so these method names can't collide with anything else you define.

This is the only setup needed. There's no connectedCallback, no subscription, no state.

Step 2 — Component is constructed

The framework creates an instance of QuickLinks. Field initializers run (there are none — the class only has methods). The template renders immediately, drawing the header bar and three cards. Each card carries an onclick attribute pointing at a method on the class:

<div onclick={handleHealthWorker} ...>

<div onclick={handleQuickGuide} ...>

<div onclick={handleCampaign} ...>

LWC sees these on… bindings and registers each as a DOM click listener on the corresponding . The methods aren't called yet — they're standing by, waiting.

This is all that happens at load time. The component is now idle, with three armed click handlers and nothing else.

Step 3 — User clicks a card

Say the user clicks the "Health Worker Connect" card. The browser fires a native DOM click event on that . The framework's listener catches it and invokes handleHealthWorker() on the component instance:

```
handleHealthWorker() {
    this[NavigationMixin.Navigate]({
        type: 'standard__webPage',
        attributes: {
            url: 'https://www.youtube.com/watch?v=your_video_id'
        }
    });
}
```

There are no parameters, no event object captured — the method ignores the click event entirely, because all the information needed for navigation is hard-coded inside the method itself. If the URL ever needed to depend on, say, the logged-in user or a record, you'd accept the event, look up the data, then call Navigate.

Step 4 — Navigate builds and dispatches a PageReference

The argument passed to Navigate is called a **PageReference** — a small object that describes a destination in a platform-neutral way:

```
{
    type: 'standard__webPage',        // what KIND of destination
    attributes: { url: '...' },       // the parameters that type expects
```

```
    state: { /* optional */ }          // optional query-string-like state
}
```

The type tells the Navigation Service which "shape" of destination this is — and each type expects different attributes:

type	What it navigates to	Required attributes
standard__webPage	An external URL (opens in a new tab in Lightning Experience)	url
standard__recordPage	A Salesforce record detail page	recordId, objectApiName, actionName
standard__objectPage	An object's list view or new-record page	objectApiName, actionName (list or new)
standard__navItemPage	A custom tab	apiName
comm__namedPage	A named page in an Experience Cloud site	name

In your three handlers, all are standard__webPage because each card points at an external YouTube URL. The Navigation Service does not actually visit the URL itself — it tells the host (Lightning Experience, Experience Cloud, or the mobile app) *"the user wants to go here; you handle the navigation."*

Under the hood, Navigate does roughly this:

```
this[NavigationMixin.Navigate](pageRef)
    │
    ├── dispatches a CustomEvent named 'lightning__navigation'
    │   with the PageReference as its detail
    │
    └── the event bubbles up through the DOM until it reaches
        the Lightning Navigation Service running at the host level
```

This is the key indirection — your component never says *"open this URL"* directly. It hands a PageReference to the platform, and the platform decides what that means in the current context.

That indirection is what makes navigation work consistently across Lightning Experience desktop (open a new browser tab), the mobile app (open the in-app browser), and Experience Cloud sites (apply community routing rules) — same PageReference, three different actual behaviours.

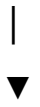
Step 5 — The Navigation Service routes the user

Once the host catches the event, it inspects the type and attributes. For standard__webPage with a url, it opens that URL — in Lightning Experience desktop, in a new browser tab; in the Salesforce mobile app, in the in-app browser.

At that moment the user leaves the page (or a new tab pops up), and as far as your component is concerned, the job is done. There is no callback, no return value, no confirmation. The Navigate method returns a Promise, but the three handlers here don't await it — fire and forget.

The flow in one picture

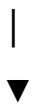
Page loads



QuickLinks constructed

NavigationMixin gives this[Navigate] and this[GenerateUrl]

no state, no Apex, no @wire



Template renders three cards

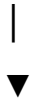
onclick handlers registered, idle

(user clicks "Health Worker Connect")

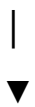
DOM click event fires on the card

Framework invokes handleHealthWorker()

```
this[NavigationMixin.Navigate]({
  type: 'standard__webPage',
  attributes: { url: '…youtube…' }
})
```



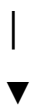
PageReference dispatched as a navigation event



Salesforce Navigation Service catches it

inspects type ('standard__webPage')

resolves to: open URL in new tab



Browser navigates / opens tab — user arrives at YouTube

How this fits alongside the two earlier components

The three components together cover the three main directions data can move in an LWC:

Component	Direction	Mechanism	Salesforce server contact
Welcome Banner	Data flows **in**	LDS via @wire(getRecord)	One UI API request, cached
Case Dashboard	Data flows **in**	Apex via @wire(getCaseCounts)	One Apex request, cached
Quick Links	Action flows **out**	NavigationMixin.Navigate	None (the platform handles routing)

Quick Links is the simplest of the three because it owns no state. The lifecycle is: render once → wait → on click, dispatch one navigation event → done. If the evaluator asks *"why doesn't this component need Apex or LDS?"* — the answer is that nothing on the page depends on Salesforce data; the URLs are static constants chosen at build time, and the navigation itself is a UI action, not a data read.

A couple of things worth knowing for the demo

The first card's URL is a placeholder (https://www.youtube.com/watch?v=your_video_id). Worth flagging — in a real demo you'd want a working link, or remove the card.

standard__webPage is fine for external links, but for navigating *inside* Salesforce — to a record page or a community page — you'd switch the type. For instance, sending the user to a specific Outreach Program record would be:

this[NavigationMixin.Navigate]({

```
    type: 'standard__recordPage',
    attributes: {
        recordId: this.programId,
        objectApiName: 'Outreach_Program__c',
        actionName: 'view'
    }
});
```

If you ever need the URL itself rather than performing the navigation (say, to populate an <a href>), the mixin also gives you this[NavigationMixin.GenerateUrl](pageRef), which returns a Promise resolving to the URL string — useful for "open in new tab" buttons or for sharing links.

And one design improvement worth mentioning if probed: hard-coding URLs in three near-identical methods is fine for three links, but if Quick Links grew to ten cards, you'd refactor to drive it from a data array (or Custom Metadata) and iterate with for:each, where each click handler reads event.currentTarget.dataset.url. That would make the component data-driven without changing the navigation mechanism — still NavigationMixin.Navigate with standard__webPage, just with a dynamic URL.

MY Patients

Here's the data flow for the My Patients component — by far the richest of the four, because it has two distinct screens, two Apex calls, a modal, and meaningful client-side data transformation.

The pieces in play

1. **Apex controller** — HealthConnectController with two @AuraEnabled(cacheable=true) methods: getMyPatients() and getPatientCases(contactId).
2. **Imperative Apex** — the JS calls them with .then()/.catch(), not with @wire. This is a deliberate choice (more on why below).
3. **State machine in @track properties** — three flags (isLoading, isCasesLoading, showCases, showCaseDetail) decide which view the template draws.
4. **Three data arrays / objects** — patients (list view), cases (drill-down), selectedCase (modal).
5. **Transformations** — the JS reshapes the raw Apex result before storing it (computed priority, formatted date, flattened owner alias).
6. **Getters** — derived numbers and booleans for the template (totalPatients, noCases, openCaseCount, caseSummary, etc.).
7. **Template** — three branches gated by the flags: list → cases → modal.

This is the first component in the set that holds proper state. The flow has *phases*: load the list, drill into a patient, drill into a case, dismiss, drill into another. Let's walk each phase.

Phase 1 — Page loads, list view appears

@track isLoading = true;

@track patients = [];

@track showCases = false;

// ... all other flags false, arrays empty

The constructor leaves everything at these defaults. So at this moment the template evaluates each branch:

- isLoading true → spinner shows.
- hasError false → no error banner.
- noPatients getter returns false (it requires !isLoading), so no "no patients" message.

- showCases false → the list view branch is selected but hidden under the spinner.

Then connectedCallback fires:

connectedCallback() { this.loadPatients(); }

This is the lifecycle hook that runs once the component is inserted in the DOM. It kicks off the first server call.

Phase 2 — loadPatients fetches the patient list

```
loadPatients() {
    getMyPatients()
        .then(result => { ... })
        .catch(error => { ... });
}
```

getMyPatients() is an imperative call to Apex — no @wire, no auto-firing. The method returns a Promise; the JS attaches success and failure handlers.

Why imperative here, not @wire?

You'll notice the Welcome Banner and Case Dashboard used @wire, but this component uses imperative calls. The reason is *sequencing*. This component needs to make a second Apex call (getPatientCases) only *after* the user clicks a patient. That's an action triggered by a user, not a reactive data binding. @wire is for "give me data continuously and re-fetch when its inputs change" — imperative is for "fetch when I ask." When you have explicit phases like this, imperative is the cleaner pattern.

(You *could* still wire getPatientCases to a $selectedPatientId reactive param — and that would work — but imperative gives you direct control over loading flags, error handling, and the order of state changes, which matters here.)

On the server: getMyPatients

```
User currentUser = [SELECT Id, Name, ContactId FROM User WHERE Id =
:UserInfo.getUserId() LIMIT 1];


List<Case> allCases = [
    SELECT ContactId FROM Case
    WHERE Assigned_Doctor__r.Name = :currentUser.Name
      AND RecordType.Id = '012dM00000D4tODQAZ'
];
```

```
Set<Id> contactIds = new Set<Id>();
for (Case c : allCases) contactIds.add(c.ContactId);


return [
    SELECT Id, Name,
        (SELECT Priority FROM Cases
         WHERE Assigned_Doctor__r.Name = :currentUser.Name
           AND RecordType.Id = '012dM00000D4tODQAZ'
         ORDER BY CreatedDate DESC)
    FROM Contact
    WHERE Id IN :contactIds
    ORDER BY Name ASC
];
```

This is a two-step query pattern:

1. **First query** — find every clinical Case where the running doctor is the Assigned_Doctor__r. We only need ContactId from each case; the JSON would otherwise be heavier.
2. **Collect unique contact Ids** into a Set<Id> (deduplicates: a patient with five cases appears five times in the case list but should be one row in the patient list).
3. **Second query** — fetch each of those Contacts, and in a *parent-to-child subquery* include the Priority of each related Case (filtered to this doctor's clinical cases). This is the standard pattern from your notes: going down from parent (Contact) to children (Cases) needs a subquery.

The result returned to the client is an array of Contact records, each with a nested Cases collection containing just Priority for each case. Something like:

```
[
  { "Id": "003xx...", "Name": "Sharma Aryan",
    "Cases": [ { "Priority": "High" }, { "Priority": "Low" } ] },
  { "Id": "003xx...", "Name": "Patel Riya",
    "Cases": [ { "Priority": "Critical" } ] }
```

]

Back on the client: transformation in .then()

This is where the component does real work:

```
this.patients = result.map(p => {
    const patientCases = p.Cases || [];
    const priorities = patientCases.map(c => c.Priority);
    let highest = 'Low';
    if (priorities.includes('Critical')) highest = 'Critical';
    else if (priorities.includes('High')) highest = 'High';
    else if (priorities.includes('Medium')) highest = 'Medium';
    return { ...p, highestPriority: highest, caseCount: patientCases.length };
});
this.isLoading = false;
```

Two enrichments are happening:

- **highestPriority** — the worst priority across all of that patient's cases, computed by checking from most severe to least severe. This drives the badge in the list. The order of the if/else if is intentional — once you find Critical, you stop.
- **caseCount** — the number of cases for this patient. Used directly in the "Total cases" column.

The { ...p, highestPriority, caseCount } syntax is a *spread* — copy all original fields of p into a new object, then add the two derived fields. The original Apex result isn't mutated; a new array of new objects is produced. This matters because LWC reactivity is reference-based — assigning this.patients = result.map(...) (a new array) triggers a re-render; mutating in place wouldn't.

Then isLoading = false. The template re-evaluates:

- isLoading false → spinner branch closes.
- noPatients — true if the array is empty, false otherwise.
- showCases false → list view branch opens.

The table renders by iterating patients with for:each={patients} for:item="p", drawing one <tr> per patient.

Important — data- attributes carry context to the click

```
<a onclick={handlePatientClick} data-id={p.Id} data-name={p.Name}>{p.Name}</a>
```

Each row's link carries the patient's Id and Name as data- attributes. This is the standard pattern for "remembering" which row was clicked when the same handler is used for many rows.

Error path

If the Promise rejects, the .catch runs:

```
.catch(error => {
    this.isLoading = false;
    this.hasError = true;
    this.errorMessage = error.body ? error.body.message : 'Unable to load patients.';
    console.error('Error loading patients:', error);
});
```

The error.body.message shape comes from Apex — when an AuraHandledException (or any Apex exception) is thrown, the framework wraps it as { body: { message: '...' } }. The fallback string covers the case where the error is a JS or network error without that shape.

Now hasError = true and the error banner appears at the top.

Phase 3 — User clicks a patient

The user clicks a row. The handler runs:

```
handlePatientClick(event) {
    const patientId   = event.currentTarget.dataset.id;
    const patientName = event.currentTarget.dataset.name;
    this.selectedPatientId   = patientId;
    this.selectedPatientName = patientName;
    this.showCases       = true;
    this.isCasesLoading  = true;
    this.cases = [];
    // then fetch...
}
```

event.currentTarget is the element the listener is attached to (the <a> or <lightning-button>); dataset.id pulls out the data-id attribute as a string. Two patient details are captured into state.

Three flags flip in one tick:

- showCases = true — switches the template from list view to cases view.

- isCasesLoading = true — shows a spinner inside the cases view.
- cases = [] — clears any cases from a previous patient drill-down.

After this batch of assignments, the framework re-renders. The list disappears, the cases view appears with a "Back to patients" button and a spinner. The second Apex call now fires:

```
getPatientCases({ contactId: patientId })
    .then(result => { ... })
    .catch(error => { ... });
```

The argument { contactId: patientId } matches the Apex method signature getPatientCases(Id contactId). Apex parameter names map to JS property names exactly.

On the server: getPatientCases

```
return [
    SELECT Id, CaseNumber, Subject, Status, Priority,
        CreatedDate, ClosedDate, Description, Origin,
        Owner.Alias, IsClosed, Condition_Category__c
    FROM Case
    WHERE Assigned_Doctor__r.Name = :currentUser.Name
      AND RecordType.Id = '012dM00000D4tODQAZ'
      AND ContactId = :contactId
    ORDER BY CreatedDate DESC
];
```

Three filters in the WHERE: this doctor's cases, of the Clinical record type, for the chosen patient. Owner.Alias uses child-to-parent dot notation — walking from Case up to its Owner User record, grabbing the Alias field. The result is an array of Case records, each with a nested Owner: { Alias: 'akuma' } object.

Transformation, take two

```
this.cases = result.map(c => ({
    ...c,
    formattedDate: c.CreatedDate
        ? new Date(c.CreatedDate).toLocaleDateString('en-IN',
            { day: 'numeric', month: 'short', year: 'numeric' })
        : '—',
```

```
    ownerAlias: c.Owner ? c.Owner.Alias : '—'
}));
this.isCasesLoading = false;
```

Two derived fields:

- **formattedDate** — the raw CreatedDate is an ISO string ("2026-04-12T10:30:00.000+0000"); the template needs "12 Apr 2026". Done client-side rather than in Apex so the formatting respects locale and doesn't waste server CPU.
- **ownerAlias** — the raw shape is c.Owner.Alias, but if the Case has no Owner the c.Owner object is missing entirely (not just empty), so c.Owner.Alias would throw Cannot read property 'Alias' of undefined. Flattening to c.ownerAlias lets the template read it safely as {c.ownerAlias}.

The '—' em-dash is the placeholder shown for missing values. Small UX touch.

isCasesLoading = false flips, the spinner clears, and the cases table renders.

Derived state at this point

Now several getters compute things from cases:

```
get noCases()       { return !this.isCasesLoading && this.cases.length === 0; }
get openCaseCount() { return this.cases.filter(c => !c.IsClosed).length; }
get hasOpenCases()  { return this.openCaseCount > 0; }
get caseSummary()   { const open = this.openCaseCount;
                      return open + ' open · ' + (this.cases.length - open) + ' closed'; }
```

hasOpenCases shows the warning banner ("3 open case(s) require your attention"). caseSummary populates the badge in the card header ("3 open · 7 closed"). These are recomputed every render.

Phase 4 — User clicks a case row

```
handleCaseClick(event) {
    const caseId = event.currentTarget.dataset.id;
    const found = this.cases.find(c => c.Id === caseId);
    if (found) {
        this.selectedCase = found;
        this.showCaseDetail = true;
    }
```

```
}
```

Notice — **no Apex call** here. The case data is already in this.cases from Phase 3; Array.find() looks up the right object by Id and stores it as selectedCase. showCaseDetail flips on, and the modal renders.

This is an important optimisation. You queried all the case fields you need (Subject, Status, Description, Owner.Alias, Origin…) in getPatientCases, so opening the modal is purely a client operation. If the modal needed extra fields that weren't queried, you'd make a third Apex call here — but it doesn't.

The modal template binds to selectedCase.CaseNumber, selectedCase.Status, selectedCase.formattedDate, selectedCase.ownerAlias — including the derived fields you added during the transformation, which is why doing the transformation in .then() rather than inline in the template pays off.

Phase 5 — User dismisses the modal

```
closeCaseDetail() {
    this.showCaseDetail = false;
    this.selectedCase = {};
}
```

The modal closes; selectedCase is reset to an empty object. The user is back on the cases table for the same patient. No data fetch.

Phase 6 — User clicks "Back to patients"

```
goBackToPatients() {
    this.showCases = false;
    this.showCaseDetail = false;
    this.cases = [];
    this.selectedPatientName = '';
    this.selectedPatientId = '';
    this.hasError = false;
}
```

The flags reset and the cases array is cleared. The template switches back to the list view — and because patients was never touched, the original list is still there, no reload needed.

This is the payoff of the imperative pattern: the cases data is *transient* state tied to the drill-down, not something the framework keeps re-fetching. You loaded patients once, drilled in and out as many times as you want, and the patient list is exactly as it was.

The flow in one picture

Page loads

constructor — defaults: isLoading=true, all arrays empty, all flags false

connectedCallback() → loadPatients()

imperative getMyPatients() ─► Apex on server

1. query running User
2. query Cases assigned to me (clinical RT) → unique ContactIds
3. query Contacts IN those Ids + (Cases subquery for Priority)

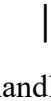
.then(result)

transform each Contact: + highestPriority, + caseCount

this.patients = […]; isLoading = false

▼ re-render: spinner gone, table of patients shown

▼ USER clicks a patient row (data-id, data-name)



handlePatientClick(event)

capture id+name, showCases=true, isCasesLoading=true, cases=[]

▼ re-render: list hidden, cases card with spinner shown

imperative getPatientCases({ contactId }) ─► Apex on server

query Cases for this doctor + RT + ContactId, with Owner.Alias

▼

.then(result)

transform each Case: + formattedDate, + ownerAlias

this.cases = […]; isCasesLoading = false

▼ re-render: cases table + warning banner if open cases

▼ USER clicks a case row

handleCaseClick(event)

find case in this.cases (no server call); set selectedCase, showCaseDetail=true

▼ re-render: modal opens over the cases table

▼ USER closes modal → closeCaseDetail()

showCaseDetail=false, selectedCase={}; modal closes

▼ USER clicks Back → goBackToPatients()

showCases=false, cases=[], all selection reset

list view shows again, patient list intact

How this stacks up against the other three

The shape of each component's data flow is now clear:

Component	Server calls	State complexity	Why this pattern
Welcome Banner	1 wire to LDS	None — wire owns everything	Single record, fields known up front
Quick Links	0	None — pure actions	No data, just navigation

Component	Server calls	State complexity	Why this pattern
Case Dashboard	1 wire to Apex	Flat — six counters	One snapshot of aggregate numbers
My Patients	**2 imperative calls**	**State machine — three views, three drill-down state vars**	**Phased flow driven by user clicks**

A few things worth knowing for the evaluator:

Why @track is still here. Modern LWC makes plain class fields reactive automatically — @track is strictly needed only for deep mutation. In this code, @track patients = [] and then this.patients = result.map(...) works fine without @track (because it's a whole reassignment). The @track decorators here are belt-and-braces — they don't hurt, but they're not all required. If asked, the honest answer is "kept for clarity and to handle any in-place mutations safely."

The brittle bits. Same as the Case Dashboard: hard-coded RecordType Id ('012dM00000D4tODQAZ') won't survive a deployment to another org, and matching Assigned_Doctor__r.Name = :currentUser.Name is name-based rather than Id-based (two doctors with the same name would collide). If asked "what would you improve" — query the RecordType Id dynamically and switch the match to Assigned_Doctor__c = :currentUser.ContactId.

Why two Apex methods, not one. You could fetch every patient with every case in a single query, but the payload would be heavy if a doctor has many patients each with many cases — and most users only drill into one or two. Splitting it means the list loads fast and the detail loads on demand. This is the "load what's visible" principle that keeps Lightning pages responsive.

Why the second click (case modal) makes no Apex call. All Case fields needed for the modal were already requested in getPatientCases. Reading from in-memory state is essentially free; making another round trip would be a wasted server call. This is a deliberate optimisation, and a good thing to point out in the demo as "intentional client-side state management."

Featured Topics

Here's the data flow for the Featured Topics component — a Knowledge Base browser with a three-view drill-down across topic → article list → article detail.

The pieces in play

1. **Apex controller** — FeaturedTopicsController with two @AuraEnabled(cacheable=true) methods reading from Salesforce Knowledge.
2. **Imperative Apex calls** — .then()/.catch() triggered by user clicks (no @wire).
3. **State machine in @track flags** — showArticleList, showArticleDetail decide which of three views the template draws.
4. **Three data stores** — no patient-style data initially; just articles (list) and selectedArticle (detail).
5. **Knowledge object model** — Knowledge__kav (Knowledge Article Version) is the queryable object behind Salesforce Knowledge.
6. **Map-based DTOs** — the Apex returns Map<String,String> rather than the raw sObject, flattening the data for the LWC.
7. **<lightning-formatted-rich-text>** — renders the article's HTML body safely.

The shape is very similar to My Patients (two Apex calls, drill-down), but with one important twist: **the first view doesn't fetch any data at all**. The three topic cards are hardcoded in the markup. Let me trace each phase.

Phase 1 — Page loads, three topic cards appear

@track showArticleList = false;

@track showArticleDetail = false;

@track articles = [];

@track selectedArticle = {};

All flags false, arrays empty. The template evaluates:

- showArticleList false AND showArticleDetail false → View 1 (the three cards) is selected.
- No spinner, no error banner.

The three cards are entirely static HTML — Camp Information, Health Tips, Patient FAQ — each with its own colour gradient, icon, blurb, and "Read More →" button. No Apex call has happened yet. Loading this view costs zero server time.

Each button carries a data-topic attribute:

<button data-topic="Camp Information" onclick={handleTopicClick}>

<button data-topic="Health Tips" onclick={handleTopicClick}>

<button data-topic="Patient FAQ" onclick={handleTopicClick}>

This is the *category name* the back-end uses to filter articles. The component itself never has to look up category Ids — the human-readable name is the join key.

Phase 2 — User clicks a topic

The user clicks "Health Tips". The handler runs:

```
handleTopicClick(event) {
    const topic = event.currentTarget.dataset.topic;   // "Health Tips"
    this.currentTopic = topic;
    this.showArticleList = true;
    this.showArticleDetail = false;
    this.isLoading = true;
    this.hasError = false;
    this.articles = [];

    getArticlesByDataCategory({ categoryName: topic })
        .then(result => { ... })
        .catch(error => { ... });
}
```

Several flags flip in one synchronous batch:

- currentTopic = "Health Tips" — shown in the View 2 header.
- showArticleList = true — template branches from View 1 to View 2.
- showArticleDetail = false — defensive, in case the user came back through a weird path.
- isLoading = true — show the spinner inside View 2.
- articles = [] — clear any leftover articles from a previous topic.

The framework batches these into one re-render: View 1 disappears, View 2 appears with a spinner. Then the imperative Apex call fires.

On the server: getArticlesByDataCategory

```
List<Knowledge__kav> articles = [
    SELECT KnowledgeArticleId, Title, Summary__c, UrlName, Body__c
    FROM Knowledge__kav
    WHERE PublishStatus = 'Online'
      AND Language = 'en_US'
      AND IsLatestVersion = true
      AND RecordType.Name = :categoryName
    ORDER BY Title ASC
    LIMIT 20
];
```

A few things matter here, and they're specific to Salesforce Knowledge:

Knowledge__kav — the *Knowledge Article Version* object. It's important to understand this isn't quite a normal sObject. Every published article has multiple versions (draft, published, archived), each as a separate Knowledge__kav row. The shared identity across versions is KnowledgeArticleId — that's the stable Id you reference from outside.

PublishStatus = 'Online' filters for the published version (as opposed to Draft or Archived). Without this you'd include drafts in progress and old archived ones.

IsLatestVersion = true ensures you only get the most recent published version of each article — if an article has been published, then republished after edits, both versions are Online but only the latest is current.

Language = 'en_US' scopes to English. Knowledge supports multi-language, so an article translated into Hindi and Tamil would appear as separate Knowledge__kav rows with the same KnowledgeArticleId.

RecordType.Name = :categoryName — here's the clever part. In this project, Record Types are being used as the "category" of an article. Camp Information, Health Tips, Patient FAQ are three Record Types on Knowledge. Clicking the "Health Tips" card sends that exact string to the WHERE clause via child-to-parent dot notation up to the RecordType parent.

LIMIT 20 — a hard cap. Reasonable for a community portal where articles per category stay small; if it ever grew, you'd add pagination.

Map-based DTO — why not return the sObject?

After the query, the code does this:

```
for(Knowledge__kav a : articles) {
```

```
    Map<String,String> m = new Map<String,String>();
    m.put('id',     a.KnowledgeArticleId);
    m.put('title',  a.Title);
    m.put('summary', a.Summary__c != null ? a.Summary__c : '');
    m.put('body',    a.Body__c    != null ? a.Body__c    : '');
    result.add(m);
}
return result;
```

Instead of returning List<Knowledge__kav>, the code flattens each record into a Map<String,String>. Three reasons this is deliberate:

1. **Stable client-side keys.** The map keys (id, title, summary, body) are lowercase and consistent. The LWC reads a.id, a.title — clean and decoupled from Salesforce field API names. If the field Summary__c were ever renamed, only the Apex layer changes; the LWC keeps reading a.summary.
2. **KnowledgeArticleId → id rename.** The article's stable identity is KnowledgeArticleId, not Id. By mapping it to a friendly key id, the LWC doesn't need to know that quirk.
3. **Null coalescing.** a.Summary__c != null ? a.Summary__c : '' ensures the LWC never gets undefined — every map always has all four keys with at least an empty string.

The payload back to the browser looks like:

```
[
  { "id": "kAxx...", "title": "Wash hands often", "summary": "Quick tip…", "body": "<p>HTML…</p>" },
  { "id": "kAxx...", "title": "Eat seasonal fruit", ...}
]
```

Back on the client: .then()

```
.then(result => {
    this.articles = result;
    this.isLoading = false;
})
```

No transformation needed — the Apex already shaped the data well. Assign and flip the flag. The template re-evaluates:

- isLoading false → spinner closes.
- noArticles getter: !isLoading && articles.length === 0 && !hasError. If the topic has no published articles, this is true, and the dashed "No articles found" empty state shows. Otherwise the article cards render via <template for:each={articles} for:item="a">.

Each article card is a horizontal row with title + summary + a "Read →" button:

<button data-id={a.id} onclick={handleArticleClick}>Read →</button>

The data-id carries the article's KnowledgeArticleId — captured here so the next handler knows what to load.

Empty state vs error state

These two getters look similar but distinguish three cases:

```
get noArticles() {
    return !this.isLoading
        && this.articles.length === 0
        && !this.hasError;
}
```

- **Loading** → spinner.
- **Error** → red error banner.
- **No articles but no error** → dashed "No articles" panel.
- **Articles** → card list.

The exclusion && !this.hasError is critical — without it, an error state would *also* show "no articles found", which is misleading.

Phase 3 — User clicks an article

```
handleArticleClick(event) {
    const articleId = event.currentTarget.dataset.id;
    this.isDetailLoading = true;
    this.showArticleDetail = true;
    this.selectedArticle = {};

    getArticleDetail({ articleId: articleId })
        .then(result => { ... })
```

```
        .catch(error => { ... });
}
```

Three flags flip:

- isDetailLoading = true — spinner inside View 3.
- showArticleDetail = true — template branches from View 2 to View 3.
- selectedArticle = {} — empty out any leftover article from a previous read.

Note showArticleList is *not* reset to false. That's because the parent guards in the template are nested:

```
<template if:false={showArticleList}>     <!-- View 1: outer guard -->
  <template if:false={showArticleDetail}>
      ... three cards ...
  </template>
</template>

<template if:true={showArticleList}>      <!-- View 2: outer guard -->
  <template if:false={showArticleDetail}>
      ... article list ...
  </template>
</template>

<template if:true={showArticleDetail}>    <!-- View 3 -->
  ... article detail ...
</template>
```

So View 2 and View 3 can both be "active" in state, but only View 3 renders because its outer guard wins. That trick is what lets you "Back to Articles" from View 3 by just flipping showArticleDetail back to false — showArticleList is still true, so View 2 reappears with its existing articles still in memory. **No re-fetch needed.**

Then unlike My Patients...

Wait — in My Patients, clicking a row also avoided a second Apex call because the case data was already in memory. Here, the code **does** make a second call. Why the difference?

Look at what was queried in the list step:

SELECT KnowledgeArticleId, Title, Summary__c, UrlName, Body__c

Body__c is already in the list result. So technically the same trick *would* work — find the article in this.articles and use it directly. But the code calls getArticleDetail anyway. Two possible reasons this is intentional:

1. **Defensive design.** The list method might evolve to skip Body__c for performance (rich-text bodies are heavy), at which point you'd need the detail call. Keeping the detail call from the start means a future optimisation doesn't break the UI.
2. **Symmetry with the API.** A "detail" endpoint is conceptually distinct from a "list" endpoint, regardless of whether the data overlaps.

This is something worth being honest about in the demo: it's an unnecessary call given the current queries, but it costs little because of cacheable=true (a second click on the same article would hit the LDS client cache, not the server).

On the server: getArticleDetail

```
try {
    List<Knowledge__kav> articles = [
        SELECT KnowledgeArticleId, Title, Summary__c, Body__c
        FROM Knowledge__kav
        WHERE KnowledgeArticleId = :articleId
         AND PublishStatus = 'Online'
         AND IsLatestVersion = true
        LIMIT 1
    ];

    if(!articles.isEmpty()) {
        Knowledge__kav a = articles[0];
        result.put('id',      a.KnowledgeArticleId);
        result.put('title',   a.Title);
        result.put('summary', a.Summary__c != null ? a.Summary__c : '');
        result.put('body',    a.Body__c    != null ? a.Body__c    : '');
    }
} catch(Exception e) {
```

```
    result.put('title',   'Error');
    result.put('summary', e.getMessage());
    result.put('body',    '');
}
return result;
```

A single record fetched by KnowledgeArticleId. Two design choices stand out:

The try/catch swallows the exception. Instead of letting it propagate up as an AuraHandledException (which would land in the LWC's .catch and set hasError), it returns a "success" with title: 'Error' and the message in summary. The LWC's .then is what runs — and the modal would display "Error" as the title and the exception text as the summary.

That's a debatable pattern. It avoids one error path but creates a confusing UX (the user sees an "Error" article rather than a clean banner). The cleaner approach is to throw an AuraHandledException so the LWC's existing error UI kicks in. If asked, the safer answer is "I'd refactor this to throw so the existing error banner handles it consistently."

LIMIT 1 even though the WHERE filters by primary key. Both KnowledgeArticleId and the IsLatestVersion = true should already narrow to one row. LIMIT 1 is belt-and-braces protection — and tells the query optimiser to stop after one match.

Detail .then() and rendering

```
.then(result => {
    this.selectedArticle = result;
    this.isDetailLoading = false;
})
```

The whole map becomes selectedArticle. The template re-renders:

```
<h2>{selectedArticle.title}</h2>

<template if:true={selectedArticle.summary}>
    <p>{selectedArticle.summary}</p>
</template>

<template if:true={selectedArticle.body}>
    <lightning-formatted-rich-text value={selectedArticle.body}></lightning-formatted-rich-text>
```

</template>

The key component here is <lightning-formatted-rich-text>. The article Body__c is rich text — HTML with formatting tags like <p>, <ul>, , possibly inline images. You can't just write {selectedArticle.body} because the template would escape the HTML and the user would see literal <p> tags as text.

lightning-formatted-rich-text solves this safely: it parses the HTML and renders it as formatted content, but it strips out anything unsafe (scripts, event handlers, iframes), so it can't be exploited by malicious article authors. It's the standard pattern for displaying user-authored rich text in Salesforce.

There's also a graceful fallback:

```
<template if:false={selectedArticle.body}>
    <template if:false={selectedArticle.summary}>
        <div>No content available for this article.</div>
    </template>
</template>
```

If both summary and body are empty, show an amber "no content" notice. This wouldn't trigger normally because the Apex coerces nulls to " — and if:true={value} is false for an empty string. So this kicks in for empty articles, which is the right behaviour.

Phase 4 — User navigates back

There are two back paths, and they differ in scope:

```
goBackToList() {
    this.showArticleDetail = false;
    this.selectedArticle = {};
    this.hasError = false;
    this.isDetailLoading = false;
}
```

goBackToList flips only the detail flag. articles is preserved. The template falls back to View 2 (since showArticleList is still true), and the user sees the article list exactly as they left it. **No Apex call.**

```
goBackToTopics() {
    this.showArticleList = false;
    this.showArticleDetail = false;
```

```
    this.currentTopic = '';
    this.articles = [];
    this.selectedArticle = {};
    this.hasError = false;
    this.isDetailLoading = false;
    this.isLoading = false;
}
```

goBackToTopics resets everything — clearing articles, current topic, and all flags. Both outer guards become false again, View 1 reappears with the three topic cards.

This is the right design. Going back to the list keeps state; going back to topics throws everything away (since the topic itself is changing).

The flow in one picture

```
Page loads
    │
    ▼
constructor — all flags false, arrays empty
    │
    ▼
View 1 renders — three static topic cards (no Apex)
    │
    ▼  USER clicks "Health Tips" card
    │
handleTopicClick(event)
  capture topic, showArticleList=true, isLoading=true
  articles=[]
    │
    ▼  re-render: View 1 hidden, View 2 shows spinner
    │
imperative getArticlesByDataCategory({ categoryName: 'Health Tips' })
  ─► Apex on server
```

Query Knowledge__kav WHERE Online + Latest + RecordType.Name

Build List<Map<String,String>> with stable keys

│

▼

.then(result) — this.articles = result; isLoading=false

│

▼ re-render: article cards or "No articles" state

│

▼ USER clicks an article's "Read →"

│

handleArticleClick(event)

capture articleId, isDetailLoading=true, showArticleDetail=true

selectedArticle={}

│

▼ re-render: outer guard switches to View 3, spinner shown

│

imperative getArticleDetail({ articleId })

─► Apex on server

Query single Knowledge__kav by KnowledgeArticleId

Return Map<String,String>

│

▼

.then(result) — this.selectedArticle = result; isDetailLoading=false

│

▼ View 3: title + summary box + rich-text body via

<lightning-formatted-rich-text>

│

▼ USER clicks "Back to Articles" → goBackToList()

showArticleDetail=false (only)

articles still in memory → View 2 instantly reappears

|

▼ USER clicks "Back to Topics" → goBackToTopics()

everything resets → View 1 reappears

How this stacks up against the four siblings

The five components now span a clear spectrum of complexity:

Component	Data flow	View structure	Server calls
Quick Links	Outbound (navigation only)	One view	0
Welcome Banner	Inbound, reactive	One view	1 wire (LDS)
Case Dashboard	Inbound, reactive	One view	1 wire (Apex)
My Patients	Inbound, imperative, drill-down	3 views (list / detail / modal)	2 imperative
Featured Topics	**Inbound, imperative, two-level drill-down**	**3 views (topics / list / detail)**	**2 imperative**

A few specific things worth knowing for the demo:

Why Knowledge__kav specifically. Salesforce Knowledge has its own object model — KnowledgeArticleVersion (queryable as Knowledge__kav) is the version object; KnowledgeArticle is the master that holds the versions. You always query __kav for content and filter by PublishStatus, IsLatestVersion, and Language — that's a standard pattern an evaluator will recognise.

Map-based DTO vs returning sObject. This is a defensible design choice: it decouples the LWC from the Salesforce field API names, normalises null values, and renames KnowledgeArticleId to id. The trade-off is you can't use the wired LDS cache as well as you could for plain sObjects, and cacheable=true works on the method-level instead of the record-level. For this read-only browser, it's fine.

without sharing. The controller is without sharing — meaning record-level sharing is ignored. That's appropriate here because Knowledge articles are typically public (or at least visible to all logged-in users), and you want anyone hitting the portal to see them regardless of their Contact's sharing rules. Worth flagging if asked: it's an intentional decision, not a security gap.

The hardcoded category names are a coupling point. Three places reference "Camp Information", "Health Tips", "Patient FAQ": the template's data-topic attributes, and the

Knowledge Record Type names. If someone renames a Record Type in Setup, the cards silently break (an empty article list). For interview robustness: "I'd improve this by fetching available Data Categories or Record Types from Apex on load and rendering the cards dynamically."

<lightning-formatted-rich-text> is the safe HTML-rendering pattern. That's the answer to "how do you display HTML from the database without an XSS hole" — use this component, which whitelists safe tags and strips scripts.

The swallowed exception in getArticleDetail. Be honest about it — the cleaner pattern is throw new AuraHandledException(e.getMessage()) so the existing error banner handles it consistently. It's a small refactor worth pointing out.

Active Campaigns

Here's the data flow for the Active Campaigns component — the simplest of the data-driven ones, since it has just one view and one Apex call.

The pieces in play

1. **Apex controller** — ActiveCampaignsController.getActiveCampaigns() returns a list of upcoming health-camp programs.
2. **Imperative call from connectedCallback** — fired once when the component is inserted in the DOM.
3. **Three state pieces** — campaigns (the data), isLoading, hasError/errorMessage (the flags).
4. **Two getters** — totalCampaigns (a label) and noCampaigns (an empty-state predicate).
5. **Template** — four branches for spinner / error / empty / cards, all selected by the flags.

There is no drill-down here. The user lands on the page, sees a grid of cards, and that's the journey. So the data flow is essentially linear: load → transform on server → render.

Step 1 — Component is constructed

@track campaigns = [];

@track isLoading = true;

@track hasError = false;

@track errorMessage = '';

Defaults: empty array, isLoading = true, no error. At this instant the template evaluates:

- isLoading true → spinner branch active.
- hasError false → no red banner.
- noCampaigns getter: !isLoading && campaigns.length === 0 && !hasError — false because isLoading is true.
- The card grid branch (if:false={isLoading} → if:false={noCampaigns}) is gated off.

So the user sees the header ("Active Campaigns — Upcoming health camps near you", with a "0 active" pill), then a spinner underneath. The header renders immediately because it's outside any <template if:> guard — only the dynamic content waits.

Note {totalCampaigns} resolves to "0 active" at this instant (empty array length + the string). It updates to "5 active" once data arrives.

Step 2 — connectedCallback fires

```
connectedCallback() {
    this.loadCampaigns();
}
```

connectedCallback runs once when the component is inserted in the DOM — after the constructor, before the first render is fully committed to the user's screen. It's the standard place to kick off "load my data" work.

A subtle but important point: this is **imperative** Apex, not @wire. Why?

For a component like this where the data is *time-sensitive* (active campaigns end, new ones start), @wire with its aggressive client-side cache can be a liability — a stale cached result could keep showing yesterday's data. Imperative gives explicit control: fetch on connectedCallback, refetch on a user action if you ever add a refresh button. There's also no reactive parameter that would naturally re-trigger a wire here — the query has no inputs.

(That said, getActiveCampaigns is still marked cacheable=true on the server, so even the imperative call will hit the LDS client cache for repeat calls within a session. The choice is about *how* you fetch, not about whether to cache.)

Step 3 — loadCampaigns calls Apex

```
loadCampaigns() {
    getActiveCampaigns()
        .then(result => { ... })
        .catch(error => { ... });
}
```

No parameters — the method needs none, since "active" is defined entirely server-side. The Promise pattern is now familiar from My Patients and Featured Topics.

On the server: the query

```
List<Outreach_Program__c> campaigns = [
    SELECT Id, Name,
        Start_Date__c, End_Date__c,
        Status__c,
        Region__r.Name,
        Target_Patients__c,
        Program_Type__c
```

```
    FROM Outreach_Program__c
    WHERE Status__c = 'Active'
      AND End_Date__c >= TODAY
    ORDER BY Start_Date__c ASC
    LIMIT 10
];
```

A few things to walk through:

Two filters in the WHERE. Status__c = 'Active' is the manual flag the admin sets. End_Date__c >= TODAY is the safety net — even if an admin forgets to flip Status to Inactive after a camp ends, the date check still excludes it. Together they answer the question "what's actually running or about to run." TODAY is a SOQL date literal, evaluated server-side; you don't pass it from the LWC.

Region__r.Name — child-to-parent dot notation. The Outreach Program has a lookup to Region (the custom backbone object that drives sharing in NeelamCare); rather than getting back the Region's Id and doing a second query, the field is read up through the relationship and returns just the Region's display name. This is the same direction-rule from your notes: going up uses dot notation, going down uses a subquery.

ORDER BY Start_Date__c ASC — soonest first. The card grid reads in order, so the campaign starting tomorrow appears top-left, and the one in three weeks appears further down.

LIMIT 10 — a hard cap. A community portal shouldn't choke if dozens of camps are active; ten is plenty for the visible grid, and you'd add pagination if it ever became a real constraint.

Map-based DTO with formatting

```
for(Outreach_Program__c c : campaigns) {
    Map<String,String> m = new Map<String,String>();
    m.put('id',     c.Id);
    m.put('name',   c.Name);
    m.put('status', c.Status__c != null ? c.Status__c : '');
    m.put('region', c.Region__r != null ? c.Region__r.Name : '');
    m.put('campType', c.Program_Type__c != null ? c.Program_Type__c : '');
    m.put('targetPatients',
        c.Target_Patients__c != null
            ? String.valueOf(c.Target_Patients__c.intValue())
```

```
            : '');
        m.put('startDate', c.Start_Date__c != null ? c.Start_Date__c.format() : '');
        m.put('endDate',   c.End_Date__c   != null ? c.End_Date__c.format()   : '');
        result.add(m);
    }
    return result;
```

Same DTO pattern as Featured Topics — flatten the sObject into a Map<String,String> with stable keys. Three details specific to this method matter:

Server-side date formatting. c.Start_Date__c.format() converts the Date into the user's locale-appropriate string ("15/06/2026" in India, "6/15/2026" in the US). The reason this is done in Apex rather than client-side: every card needs the formatted date, and the format depends on the running user's locale — which Apex knows via UserInfo. Doing it once on the server means the LWC just reads c.startDate as a finished string.

This is a different choice from My Patients, which did formatting in JS with toLocaleDateString('en-IN', ...). Both are valid; the My Patients approach hardcodes Indian formatting, this one respects whatever locale the running user has.

Null-handling for the parent reference. c.Region__r != null ? c.Region__r.Name : '' — if a Program record has no Region (the lookup is blank), then c.Region__r is null. Accessing .Name on a null reference would throw. The defensive check coalesces to an empty string instead, and the card just shows an empty Region field rather than crashing the whole fetch.

Number-to-string coercion. Target_Patients__c is probably a Number field, but the map declares Map<String,String> — so the value must be a string. The chain is:

```
c.Target_Patients__c.intValue()       // Decimal → Integer
  ↓
String.valueOf(...)                   // Integer → String "200"
```

Why the .intValue() step? Number fields in Salesforce default to Decimal in Apex, even when the precision is set to 0. String.valueOf(200.0) gives "200.0" — not what you want on a card. intValue() truncates to a whole number first, so the user sees "200". Worth knowing for any "why didn't you just use String.valueOf(...)" question.

The result payload is roughly:

```
[
  { "id": "a01...", "name": "Tambaram Diabetes Camp",
    "status": "Active", "region": "Chennai South",
    "campType": "Diabetes Screening",
```

```
  "targetPatients": "200",
  "startDate": "15/06/2026", "endDate": "16/06/2026" },
 ...
]
```

Step 4 — .then() plus the template's branch logic

```
.then(result => {
  this.campaigns = result;
  this.isLoading = false;
})
```

Two assignments. No further transformation — the Apex did all the work. The framework detects both changes and schedules a re-render.

Now the template re-evaluates:

- isLoading false → spinner branch closes.
- noCampaigns getter: !isLoading && campaigns.length === 0 && !hasError.
    - If the query returned an empty array → noCampaigns is true → dashed "No active campaigns" empty state appears.
    - If campaigns came back → noCampaigns is false → card grid renders.

Meanwhile the header pill at the top updates: totalCampaigns reruns and now returns "5 active".

The card grid is rendered with <template for:each={campaigns} for:item="c">, drawing one card per campaign. Each card is purely template-driven from the map fields — the LWC has no per-card logic. Inside the card there's only one conditional:

```
<template if:true={c.targetPatients}>
  <!-- the green "Target Patients" tile -->
</template>
```

Since targetPatients is always coerced to a string ("" for missing, "200" for set), if:true={c.targetPatients} is false when empty (an empty string is falsy in LWC's truthiness check) and true otherwise. So campaigns with no target count simply omit that tile, and the card still renders cleanly.

Step 5 — Error path

If the Apex throws (or the network drops), the Promise rejects:

```
.catch(error => {
```

```
    this.isLoading = false;
    this.hasError = true;
    this.errorMessage = error.body
        ? error.body.message
        : 'Unable to load campaigns.';
    console.error('Error loading campaigns:', error);
});
```

Same recipe as the other components: turn off the spinner, flip the error flag, capture the message. error.body.message is the standard shape when Apex throws an AuraHandledException; the fallback string covers network errors.

Now hasError = true → the red banner shows. And noCampaigns returns false (because of && !this.hasError), so the empty state is correctly suppressed — the user sees only the red banner, not "no campaigns" misleading them about the cause.

The flow in one picture

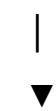
Page loads

constructor — defaults: campaigns=[], isLoading=true, no error

Template first render

header shows ("0 active" pill)

spinner visible



connectedCallback() → loadCampaigns()

imperative getActiveCampaigns() ─► Apex on server

query Outreach_Program__c WHERE Status='Active' AND End_Date >= TODAY

```
    ORDER BY Start_Date ASC LIMIT 10, with Region__r.Name
  loop: flatten each record into Map<String,String>
    - format dates server-side (locale-aware)
    - null-guard Region__r and Target_Patients__c
    - coerce Decimal to "200" via intValue + String.valueOf
  return List<Map<String,String>>
    │
    ▼
.then(result)
  this.campaigns = result
  this.isLoading = false
    │
    ▼  re-render
  header pill: "5 active"
  spinner gone
  for:each renders one card per campaign
  per-card: if targetPatients truthy → green tile
    │
    ▼
DONE — no further interactivity, no drill-down
```

How this stacks up against the other five

The series now covers six components across the full range of patterns:

Component	Server calls	View structure	Reactive update	Notable pattern
Quick Links	0	One view	None	NavigationMixin only
Welcome Banner	1 wire (LDS)	One view	Auto via wire	Schema imports + UI API
Case Dashboard	1 wire (Apex)	One view	Auto via wire	Aggregate counts + getters

Component	Server calls	View structure	Reactive update	Notable pattern
My Patients	2 imperative	3 views drill-down	Manual via flags	State machine with modal
Featured Topics	2 imperative	3 views drill-down	Manual via flags	Knowledge__kav + rich text
Active Campaigns	**1 imperative**	**One view**	**Manual via flags**	**Server-side formatting**

Active Campaigns is conceptually the cleanest example of the imperative-Apex-on-load pattern: no inputs, no drill-down, no transformations on the client, just fetch and render. The interesting work all happens in Apex — the date formatting, the null guarding, the decimal-to-int coercion.

A few things worth knowing for the demo:

Why without sharing on the controller. Campaigns should be visible to all community visitors, even guest users with restrictive sharing. without sharing ignores the running user's record access. For a "what's happening publicly" view, that's the right choice. If asked, frame it as "this is intentional — campaigns are public marketing data, not patient PHI."

Why server-side date formatting instead of client-side. A locale-aware format respects the user's settings (an admin in the UK and a doctor in India see different orderings of day/month). Doing it in Apex with c.Start_Date__c.format() lets Salesforce handle the locale logic centrally. The trade-off is that you can't easily change the format per component — but you usually don't want to.

Why a header pill that always shows even before data loads. {totalCampaigns} is a getter, not a wired value, so it runs on every render. Before data arrives it shows "0 active", then "5 active" after. Some teams hide this until loaded to avoid the flicker; others show the running total because it gives the user something to look at. Both are defensible.

The noCampaigns guard is the most important boolean in the file. Read it again: !this.isLoading && this.campaigns.length === 0 && !this.hasError. Three conditions in series, each excluding a separate concurrent state — "we're not still loading", "data really is empty", "and it's not because of an error". Without that third clause, an error would cause both the red banner and the empty state to show at once. This kind of compound boolean is what separates "works in the happy path" from "behaves correctly in every state."

Caching behaviour. cacheable=true on the Apex method means LDS will cache the result client-side for the session. If the user navigates away and back to this component, the second connectedCallback calls getActiveCampaigns() and gets the result instantly from the cache without hitting the server. That's good for performance, but a campaign added in the last few

minutes won't show up until the cache expires or the user reloads the page. For a non-realtime view of marketing data, this is fine.

Patient Immunization

Here's the data flow for the Patient Immunization component — the patient-side mirror of My Patients, with one table view and a detail modal.

The pieces in play

1. **Apex controller** — ImmunizationController.getPatientImmunizations() finds the running user's Contact and returns their vaccine records.
2. **Imperative call from connectedCallback** — fires once when the page loads.
3. **Client-side transformation** — formats dates, derives a reminder-status string, and computes whether the next dose is upcoming.
4. **Two state pieces** — records (the table data) and selectedRecord (the modal data).
5. **Two booleans for the modal** — showDetail toggles the modal; selectedRecord carries what to show.
6. **No second Apex call** — the modal reads from in-memory state, just like the case modal in My Patients.

This component sits at the intersection of two patterns you've already seen: it's a one-Apex-call-on-load (like Active Campaigns), with a click-to-modal (like My Patients). And the modal is "free" because all the detail fields were already fetched.

Step 1 — Component is constructed

@track isLoading = true;

@track records = [];

@track showDetail = false;

@track selectedRecord = {};

Defaults: empty records, isLoading = true, modal hidden. At this instant the template evaluates:

- isLoading true → spinner shows.
- hasError false → no banner.
- noRecords getter: false (because isLoading is true).
- The table branch is gated off (if:false={isLoading} → if:false={noRecords}).
- showDetail false → no modal.

The header is outside any conditional, so the user immediately sees the page title, the "Your vaccination history and upcoming doses" subtitle, and a "0 records found" pill on the right. Then a spinner underneath.

Step 2 — connectedCallback fires

```
connectedCallback() {
    this.loadRecords();
}
```

Same pattern as Active Campaigns and My Patients: load on insert. There are no inputs to the query, so reactive @wire wouldn't add anything; imperative is the cleanest fit.

Step 3 — Apex call and the "find the patient" lookup

```
loadRecords() {
    getPatientImmunizations()
        .then(result => { ... })
        .catch(error => { ... });
}
```

No parameters — the method figures everything out from UserInfo on the server.

On the server: the two-step lookup

```
Id userId = UserInfo.getUserId();

User u = [
    SELECT ContactId
    FROM User
    WHERE Id = :userId
    LIMIT 1
];

if (u.ContactId == null) {
    return new List<SObject>();
}
```

```
return [
    SELECT Id, Name, Vaccine__c, Date_Administered__c,
        Next_Due_Date__c, Dose_Number__c, Reminder_Sent__c
    FROM Immunization__c
    WHERE Patient__c = :u.ContactId
];
```

Walking through this:

UserInfo.getUserId() returns the Id of the running user — whoever opened the portal page. This is the patient, since portal users authenticate as themselves.

The User → Contact bridge. Community/portal users in Salesforce are linked to a Contact record via the ContactId field on User. That Contact represents the person in your data model — and crucially, the Immunization__c.Patient__c lookup also points to a Contact. So the chain is:

UserInfo.getUserId() → User.ContactId → Immunization__c.Patient__c

This is the standard pattern for "show *my* data" in Experience Cloud. You can't query Immunizations by User Id directly because they're not linked to Users — they're linked to Contacts.

Defensive early return. If u.ContactId is null (an internal Salesforce user accessing the portal, or a misconfigured user record), the method returns an empty list rather than throwing. Empty list flows through the same code path as "no records exist" — the user sees the "No immunization records found" empty state. No error, no crash.

The Immunization query. Pulls all the fields the table and modal need in one go: vaccine name, date administered, next due date, dose number, reminder flag. No subquery, no relationship traversal — simple flat fetch from one object filtered by patient.

Notice — no transformation in Apex. Unlike Featured Topics and Active Campaigns, this method returns the raw List<SObject> (effectively List<Immunization__c>) without flattening to a Map. The server sends back records with their native field API names:

```
[
  { "Id": "a02...", "Name": "IMM-0001",
    "Vaccine__c": "COVID-19", "Dose_Number__c": 2,
    "Date_Administered__c": "2025-03-15",
    "Next_Due_Date__c": "2026-09-15",
    "Reminder_Sent__c": true },
```

...

]

The LWC will read fields by their __c API names. That's a different design choice from the Featured Topics / Active Campaigns components, which mapped to friendly keys server-side. Both are valid; this approach is faster to write but tighter-coupled to the Salesforce schema (if a field is renamed, the LWC breaks).

Why return new List<SObject>() and not List<Immunization__c>?

A small but interesting choice. The return type signature is List<SObject>, which is the abstract base. This is technically a polymorphic return — the method *could* return any sObject type, though here it always returns Immunizations. For an @AuraEnabled method this is fine; the LWC just gets the JSON array and doesn't care about the typed wrapper. The benefit is that new List<SObject>() is a one-line empty fallback that matches the declared type without needing a typed constructor.

Step 4 — Client-side transformation

This is where the immunization component does real work — much more than Active Campaigns, less complex than My Patients:

```
this.records = result.map(r => {
    const nextDue = r.Next_Due_Date__c
        ? new Date(r.Next_Due_Date__c)
        : null;
    const today = new Date();
    const isUpcoming = nextDue && nextDue > today;


    return {
        ...r,
        formattedDateGiven:
            r.Date_Administered__c
                ? new Date(r.Date_Administered__c).toLocaleDateString('en-IN',
                    { day: 'numeric', month: 'short', year: 'numeric' })
                : '—',
        formattedNextDue:
            r.Next_Due_Date__c
```

```
            ? new Date(r.Next_Due_Date__c).toLocaleDateString('en-IN',
                { day: 'numeric', month: 'short', year: 'numeric' })
            : '—',
        reminderStatus: r.Reminder_Sent__c ? 'Sent' : 'Pending',
        isUpcoming: isUpcoming
    };
});
this.isLoading = false;
```

Four derived fields are added to each record via the spread ({...r, ...new}):

formattedDateGiven — converts the ISO date string "2025-03-15" into "15 Mar 2025" for display. The 'en-IN' locale gives day-first ordering; the options force a short-month format (Mar not March, 3/15/2025, etc). If the date is missing, an em-dash placeholder shows instead.

formattedNextDue — same treatment for the next-due-date field. Both formats match so the table reads consistently.

reminderStatus — flattens a Boolean into a friendly string. Reminder_Sent__c = true → 'Sent', otherwise 'Pending'. This is what populates the modal's "Reminder sent" detail line. The table itself uses a different approach — it reads the raw Boolean and chooses between two <lightning-badge> elements:

```
<template if:true={r.Reminder_Sent__c}>
    <lightning-badge label="Sent"></lightning-badge>
</template>
<template if:false={r.Reminder_Sent__c}>
    <lightning-badge label="Pending"></lightning-badge>
</template>
```

Both work, but the table is slightly redundant — it could read {r.reminderStatus} directly into one <lightning-badge>. Not a bug, just a small inconsistency.

isUpcoming — the most interesting derivation. It checks whether the next-due date is in the future:

```
const nextDue = r.Next_Due_Date__c ? new Date(r.Next_Due_Date__c) : null;
const today = new Date();
const isUpcoming = nextDue && nextDue > today;
```

If Next_Due_Date__c is missing entirely (no dose scheduled), nextDue is null and the short-circuit nextDue && nextDue > today evaluates to null (falsy). If the date exists and is after now, isUpcoming is true. If the date exists but is in the past (the dose is overdue or already taken), it's false.

This drives the warning banner inside the modal:

```
<template if:true={selectedRecord.isUpcoming}>
  <div class="slds-notify slds-notify_alert slds-theme_warning">
    Your next dose is due on {selectedRecord.formattedNextDue}.
    Please visit your nearest health camp.
  </div>
</template>
```

So the modal shows the amber warning *only* for future doses, not for completed ones or records with no scheduled follow-up. That's an honest piece of UX logic — patients see a call-to-action only when there's actually something to do.

Subtle gotcha — today vs nextDue timing

new Date() includes time, not just date. If Next_Due_Date__c is a Salesforce Date (no time), parsing it gives midnight UTC. Compared to new Date() at, say, 2pm local time, the comparison "is nextDue > today" can flip near midnight in edge timezones. It works fine for clear future or past dates; it's only ambiguous for "today" itself. For the immunization use case (doses scheduled days in advance), this isn't an issue, but worth knowing if asked.

isLoading = false flips, the template re-renders, the spinner clears, and the table appears.

Step 5 — The table renders

The header pill updates from "0 records found" to "12 records found" (or whatever the count is). The table iterates records with for:each and draws one row per record, reading both the raw fields ({r.Name}, {r.Vaccine__c}, {r.Dose_Number__c}) and the derived ones ({r.formattedDateGiven}, {r.formattedNextDue}).

Each row carries a data-id on the clickable elements:

```
<a onclick={handleRecordClick} data-id={r.Id}>{r.Name}</a>
<lightning-button onclick={handleRecordClick} data-id={r.Id} label="View"/>
```

Both the name link and the "View" button trigger the same handler with the same Id — giving the user two ways to open the same modal.

Step 6 — User clicks a row — modal opens

```
handleRecordClick(event) {
```

```
    const recordId = event.currentTarget.dataset.id;
    const found = this.records.find(r => r.Id === recordId);
    if (found) {
        this.selectedRecord = found;
        this.showDetail = true;
    }
}
```

Identical pattern to handleCaseClick in My Patients:

1. Read the data-id to learn *which* record was clicked.
2. Find that record in the already-loaded this.records array via Array.find().
3. Store it as selectedRecord.
4. Flip showDetail = true.

No Apex call. The modal opens with all the data — the raw fields *and* the derived fields like formattedDateGiven, reminderStatus, isUpcoming — because they were computed once during the initial transformation. This is the payoff of doing the transformation work in .then() rather than inline in the template.

The framework detects the state changes and re-renders. The modal section (gated by <template if:true={showDetail}>) appears with the backdrop dimming the table behind. Inside:

- Title: {selectedRecord.Name} — the auto-numbered record name like "IMM-0001".
- Two columns of detail labels and values.
- The conditional warning banner for upcoming doses.

The if:true={selectedRecord.isUpcoming} check on the warning banner is the only conditional inside the modal — everything else is unconditional display.

Step 7 — User dismisses the modal

```
closeDetail() {
    this.showDetail = false;
    this.selectedRecord = {};
}
```

Two assignments: hide the modal, reset the selected record to an empty object. The framework re-renders without the modal. The table is still showing because nothing about it changed.

There's no "go back" or "drill down deeper" path — this is a single-level detail. The user can open another row's modal immediately by clicking another link.

The flow in one picture

Page loads

constructor — records=[], isLoading=true, showDetail=false

Template first render

header + "0 records found" pill

spinner visible

connectedCallback() → loadRecords()

imperative getPatientImmunizations() ─► Apex on server

userId = UserInfo.getUserId()

query User → ContactId

if no ContactId → return []

else query Immunization__c WHERE Patient__c = ContactId

with Name, Vaccine, Date, NextDue, Dose, ReminderSent

.then(result)

transform each Immunization with .map():

+ formattedDateGiven (locale-formatted)

+ formattedNextDue (locale-formatted)

```
    + reminderStatus      ('Sent' / 'Pending')
    + isUpcoming          (nextDue > today?)
  this.records = transformed
  this.isLoading = false
    |
    ▼ re-render — table appears, "12 records found"
    |
    ▼ USER clicks a row (data-id captures the Id)
    |
handleRecordClick(event)
  find record in this.records by Id (no server call)
  this.selectedRecord = found
  this.showDetail = true
    |
    ▼ re-render — modal opens over the table + backdrop dims
    |
    ▼ modal reads:
      {selectedRecord.Name}            — raw
      {selectedRecord.Vaccine__c}      — raw
      {selectedRecord.formattedDateGiven}  — derived
      {selectedRecord.reminderStatus}  — derived
      if {selectedRecord.isUpcoming}   — derived → amber warning
    |
    ▼ USER clicks Close → closeDetail()
        showDetail=false, selectedRecord={}
        modal disappears, table intact
```

How this stacks up against the other six

The full series now covers seven components:

Component	Server calls	View structure	Special technique
Quick Links	0	One view	NavigationMixin only
Welcome Banner	1 wire (LDS)	One view	Schema imports
Case Dashboard	1 wire (Apex)	One view	Computed getters drive bars/ring
Active Campaigns	1 imperative	One view	Server-side date formatting
Patient Immunization	**1 imperative**	**One view + modal**	**Client-side transformation; derived isUpcoming**
My Patients	2 imperative	3 views drill-down	State-machine flags
Featured Topics	2 imperative	3 views drill-down	Knowledge__kav + rich text

Patient Immunization fills the gap between Active Campaigns (one Apex, one view, no modal) and My Patients (two Apex, three views, modal). Same data-fetching pattern, modal added on top — and crucially, the modal is "free" because no second server call is needed.

A few things worth knowing for the demo:

Why the User → Contact lookup is unavoidable. This is the question evaluators love to ask: "how does the portal know which patient's records to show?" The answer is the two-step Apex chain — UserInfo gives you the User Id, the User record has a ContactId linking to the portal user's Contact, and Immunization records are linked to that Contact. Without that bridge, you couldn't filter the query.

Why the controller is without sharing. This *could* be with sharing — if your sharing model already restricts patients to see only their own Immunizations (via OWD Private + a sharing rule), then with sharing would do the work and the WHERE clause would be a safety net. Using without sharing and relying on the WHERE clause for security is a valid alternative, but riskier — if someone reused this method elsewhere without the WHERE, they'd see everyone's records. Honest answer: "I'd switch to with sharing for defence in depth."

Hardcoded 'en-IN' locale. Active Campaigns let Salesforce pick locale on the server; this component pins to India. If the platform ever served users in other locales, you'd switch to toLocaleDateString() with no locale argument (uses the browser default) or have Apex format it like Active Campaigns does. Not a bug — just a portfolio of choices across the project.

The redundant reminderStatus field. The transformation creates reminderStatus as 'Sent'/'Pending', but the table uses two conditional <lightning-badge> elements reading the raw Boolean. Only the modal uses the derived string. Worth noting as a small refactor

opportunity: the table could read {r.reminderStatus} into one badge and drop the conditional template. Three fewer lines of HTML, same behaviour.

isUpcoming is the kind of derived-state logic that belongs client-side. "Is the date in the future" depends on *now* — which keeps moving. Computing it client-side means you don't have to refresh the page at midnight to get the right state. Apex-side computation would freeze the value at query time. (For dates far in the future or far in the past, it doesn't matter; for borderline ones, the client-side approach is right.)

Single empty-state vs error vs no-contact-Id. The noRecords getter triggers when the array is genuinely empty *and* there's no error. The Apex early return if (u.ContactId == null) return new List<SObject>() funnels into this same path — so a portal user with no Contact gets the friendly "no immunization records" message rather than a crash or a confusing error. That's the right call: from the patient's perspective, "nothing on file" is a clean and accurate message regardless of *why* the list is empty.

Raise a Support Case

Here's the data flow for the Raise Support Case component — the only one in your project that **writes** data, with form validation, a success screen, and navigation at the end.

The pieces in play

1. **@wire(getRecord)** — looks up the running user's ContactId via LDS on page load.
2. **@track form fields** — subject, description, and a parallel pair of error booleans.
3. **onchange handlers** — push input values into state and clear errors on the fly.
4. **A validate() helper** — runs before submission, sets error flags, returns a verdict.
5. **Imperative Apex call** — createSupportCase performs the DML server-side.
6. **Two view states** — form view and success view, toggled by isSubmitted.
7. **NavigationMixin** — sends the user to the portal home after success.

This is the first component you've seen that combines all three directions of data flow: data flows **in** (the wired ContactId), data flows **out** (the form submission), and **navigation** routes the user away at the end. It's the most architecturally complete piece in the set.

Step 1 — Component is constructed

extends NavigationMixin(LightningElement)

Same mixin pattern from Quick Links — NavigationMixin adds this[NavigationMixin.Navigate] to the class. Used later for the "Go to Home" button.

@track subject = '';

@track description = '';

@track isSubmitted = false;

@track isLoading = false;

@track hasError = false;

@track errorMessage = '';

@track subjectError = false;

@track descriptionError = false;

contactId;

Eight tracked state variables, plus one plain field for contactId. Notice contactId is *not* @track'd — that's a small but deliberate detail. It's set once when the wire returns, then never changes during the component's lifetime, and nothing in the template binds to it directly. So there's no reactivity needed.

At this instant the template evaluates:

- isSubmitted false → form view selected.
- isLoading false → no spinner; submit button enabled.
- hasError false → no red banner.
- subjectError and descriptionError false → no inline field errors.

The form draws immediately: header gradient, info box, two empty inputs, submit button. The user can start typing right away, even before the wire has returned with the ContactId.

Step 2 — @wire(getRecord) fires in parallel

```
@wire(getRecord, {
    recordId: userId,
    fields: [USER_CONTACT_ID]
})
wiredUser({ error, data }) {
    if (data) {
        this.contactId = data.fields.ContactId.value;
    }
    if (error) {
        console.error('Error getting user contact:', error);
    }
}
```

This is the same pattern from the Welcome Banner — userId from @salesforce/user/Id says *whose* record, USER_CONTACT_ID is the schema-imported field reference for User.ContactId. LDS issues a single UI API call, hits the client cache if the same User+ContactId fetch has happened elsewhere on the page (Welcome Banner reads different fields, so the caches don't help each other here).

When the wire returns, the function form of @wire is used (not the property form like in Welcome Banner). Why? Because the component doesn't bind the wire result to the template — it just needs to extract one field and stash it. The function form lets you do exactly that: dig into data.fields.ContactId.value and assign to this.contactId.

A subtlety: notice the assignment to this.contactId does **not** trigger a re-render. There's no @track on contactId, and nothing in the template references it. So the wire's success completes silently in the background — no visual change. This is intentional. If contactId were @track'd, the framework would do extra work for nothing.

The error branch is also notable: it logs to console but doesn't set hasError. The user can still type in the form; the failure only becomes visible when they try to submit, which we'll see in a moment.

The race condition you might worry about

What if the user types fast and clicks Submit before the wire returns? At that point this.contactId is still undefined. The Apex call would receive contactId: undefined, which Salesforce serializes as null. The Apex method would then insert a Case with ContactId = null — and that might be allowed by the schema (Case.ContactId is optional in standard Salesforce), in which case the case would be created but unlinked to any patient.

This is a real edge case that the current code doesn't guard against. The simplest fix would be in validate():

```
if (!this.contactId) {
    this.hasError = true;
    this.errorMessage = 'Still loading user info, please wait a moment.';
    return false;
}
```

Worth mentioning if asked "what would you improve."

Step 3 — User types in the Subject field

```
<input type="text" id="subject-input"
       value={subject}
       onchange={handleSubjectChange}
       maxlength="255" />
```

value={subject} binds the property to the input's value (one-way data flow from JS to DOM). onchange fires when the user finishes editing — specifically when the input loses focus *after* the value changed. Not on every keystroke. Then:

```
handleSubjectChange(event) {
    this.subject = event.target.value;
    if (this.subject.trim().length > 0) {
        this.subjectError = false;
```

```
    }
}
```

Two things happen:

1. Push the new value into state.
2. **If the field now has content, clear the error flag.** This is the "live correction" pattern — once the user fixes a previously-flagged empty field, the red error message disappears even before they hit Submit again.

It only *clears* errors here; it never *sets* them. Errors only get set inside validate(). So the user can type a wrong answer (something not actually wrong from a validation perspective) without seeing red, and only sees errors after submission.

handleDescriptionChange is the same pattern for the textarea.

Why getter-based class names instead of just toggling a class

```
get subjectFormClass() {
    return this.subjectError
        ? 'slds-form-element slds-m-bottom_medium slds-has-error'
        : 'slds-form-element slds-m-bottom_medium';
}
```

And in the template:

```
<div class={subjectFormClass}>
```

This is the LWC idiom for conditional classes. You can't write something like class="slds-form-element {subjectError ? 'slds-has-error' : "}" — LWC's {} bindings only support a property name, not an expression. So you build the full string in a getter and reference it.

slds-has-error is an SLDS class that automatically restyles child labels, inputs, and helper text in red when applied to the form element wrapper. That's why the input itself doesn't need separate red styling — the SLDS conventions handle it.

Step 4 — User clicks Submit, validation runs first

```
handleSubmit() {
    this.hasError = false;
    if (!this.validate()) {
        return;
    }
    this.isLoading = true;
```

```
    createSupportCase({ ... }).then(...).catch(...);
}
```

The handler does three things in order:

Clear any previous top-level error. If the user submitted once and got a server error, then fixed the form, the red banner from before shouldn't linger.

Validate locally first. No point hitting the server if the form is invalid. validate() returns true/false:

```
validate() {
    let isValid = true;


    if (!this.subject || this.subject.trim() === '') {
        this.subjectError = true;
        isValid = false;
    }
    if (!this.description || this.description.trim() === '') {
        this.descriptionError = true;
        isValid = false;
    }
    return isValid;
}
```

Two checks: subject and description must be non-blank after trimming. The pattern is to check **both** fields even if the first one fails — this way the user sees both error messages at once and can fix everything in one pass, instead of seeing one error, fixing it, re-submitting, then seeing the next. Small UX detail but worth pointing out.

isValid accumulates: starts true, gets flipped to false by any failure. If the function returns false, handleSubmit returns early — no spinner, no Apex call, just the inline error messages showing up.

Trigger the spinner and fire the call. isLoading = true instantly disables the submit button (via disabled={isLoading} in the template) and shows a small spinner above the divider. This prevents double-clicks creating two cases.

Step 5 — Apex DML creates the Case

```
createSupportCase({
```

```
    subject: this.subject,
    description: this.description,
    contactId: this.contactId
})
```

Three named parameters matching the Apex method signature exactly. JavaScript object → JSON → server-side Apex parameters with identical names.

On the server: lookup, build, insert

```
RecordType supportRT = [
    SELECT Id FROM RecordType
    WHERE SObjectType = 'Case'
      AND DeveloperName = 'Support_Case'
    LIMIT 1
];


Case newCase = new Case();
newCase.Subject = subject;
newCase.Description = description;
newCase.ContactId = contactId;
newCase.RecordTypeId = supportRT.Id;
newCase.Status = 'New';
newCase.Origin = 'Portal';
newCase.Priority = 'Medium';
newCase.Type = 'Support';


insert newCase;
```

Walking through this:

Dynamic RecordType lookup. This is the *right* way to handle Record Types, in contrast to the Case Dashboard and My Patients which hardcoded the Id '012dM00000D4tODQAZ'. Here the method queries by DeveloperName = 'Support_Case' — the stable, deployable identifier that doesn't change between orgs. If you deploy this to a new sandbox or to production, the lookup still works because DeveloperName is portable; the Id isn't.

Worth mentioning in the demo: "This is the pattern I'd retrofit across the project — query Record Types by DeveloperName instead of hardcoding Ids."

Builder pattern. Eight field assignments build the Case in memory. The four mandatory ones come from form input (Subject, Description, ContactId, RecordTypeId); the other four are server-supplied defaults (Status='New', Origin='Portal' tells anyone looking at the case that it came from the patient portal not email or phone, Priority='Medium' is a sensible default, Type='Support' classifies it).

Single DML. insert newCase saves to the database. This triggers the standard order of execution: system validation, before triggers, validation rules, duplicate rules, save, after triggers, assignment rules, auto-response rules, workflow, flows, etc. So if your NeelamCare org has any Case-related automation (assignment rules to route to a queue, an auto-response email to the patient, an SLA milestone), all of that fires here automatically. The LWC doesn't need to do any of it.

The try/catch wraps the right way. Unlike FeaturedTopicsController.getArticleDetail which silently swallowed the exception, this one re-throws as AuraHandledException:

```
} catch(Exception e) {
    throw new AuraHandledException(e.getMessage());
}
```

That's exactly the pattern you should follow. AuraHandledException is the type LWC understands — when it propagates to the client, the error arrives as { body: { message: '...' } }, which the LWC's .catch knows how to read.

If the catch were missing entirely, the same exception would still reach the LWC, but the message would be more verbose and include the stack trace — useful for debugging, but ugly for users to see.

The method returns void

```
public static void createSupportCase(...) {
```

The Promise on the JS side resolves with undefined. That's why the success handler is:

```
.then(() => {
    this.isLoading = false;
    this.isSubmitted = true;
})
```

No result parameter — there's nothing to use it for. The component just needs to know "did it succeed", which the Promise resolution itself signals.

You *could* design this to return the new Case Id (or the CaseNumber), and then show "Your case has been created as #00001234" on the success screen. The current design is simpler but loses that confirmation detail.

Step 6 — Success path

```
.then(() => {
    this.isLoading = false;
    this.isSubmitted = true;
})
```

Two flags flip. isLoading = false clears the spinner and re-enables the submit button (even though the form is about to disappear). isSubmitted = true flips the outer template guard.

The template re-evaluates:

```
<template if:true={isSubmitted}>
    <!-- success view -->
</template>
<template if:false={isSubmitted}>
    <!-- form view -->
</template>
```

The form view's whole DOM is torn down, and the success view's DOM is built up. The user sees the green checkmark circle, "Request Submitted Successfully!", instructions, and a "Go to Home" button.

The state of the form (subject, description, ContactId, the error flags) is still in memory — but irrelevant because nothing in the success view reads it. A small consideration: if you wanted to allow the user to file a *second* support request without reloading the page, you'd need a "Submit another" button that resets subject = '', description = '', subjectError = false, descriptionError = false, and isSubmitted = false. The current design forces the user to navigate home and come back.

Step 7 — Error path

```
.catch(error => {
    this.isLoading = false;
    this.hasError = true;
    this.errorMessage = error.body
        ? error.body.message
```

```
        : 'Something went wrong. Please try again.';
});
```

Spinner off, red banner on, error message captured. error.body.message works because Apex threw an AuraHandledException — the fallback string is for network failures or other shapes.

Important: the form view is *not* dismissed. The user is left on the same screen with their typed values intact (subject and description are still in @track state, still bound to the inputs via value={subject}), the inline errors clear if they corrected them earlier, and the red banner at the top explains what happened. They can fix and resubmit without retyping.

Step 8 — Navigation away

```
goToHome() {
    this[NavigationMixin.Navigate]({
        type: 'standard__namedPage',
        attributes: { pageName: 'home' }
    });
}
```

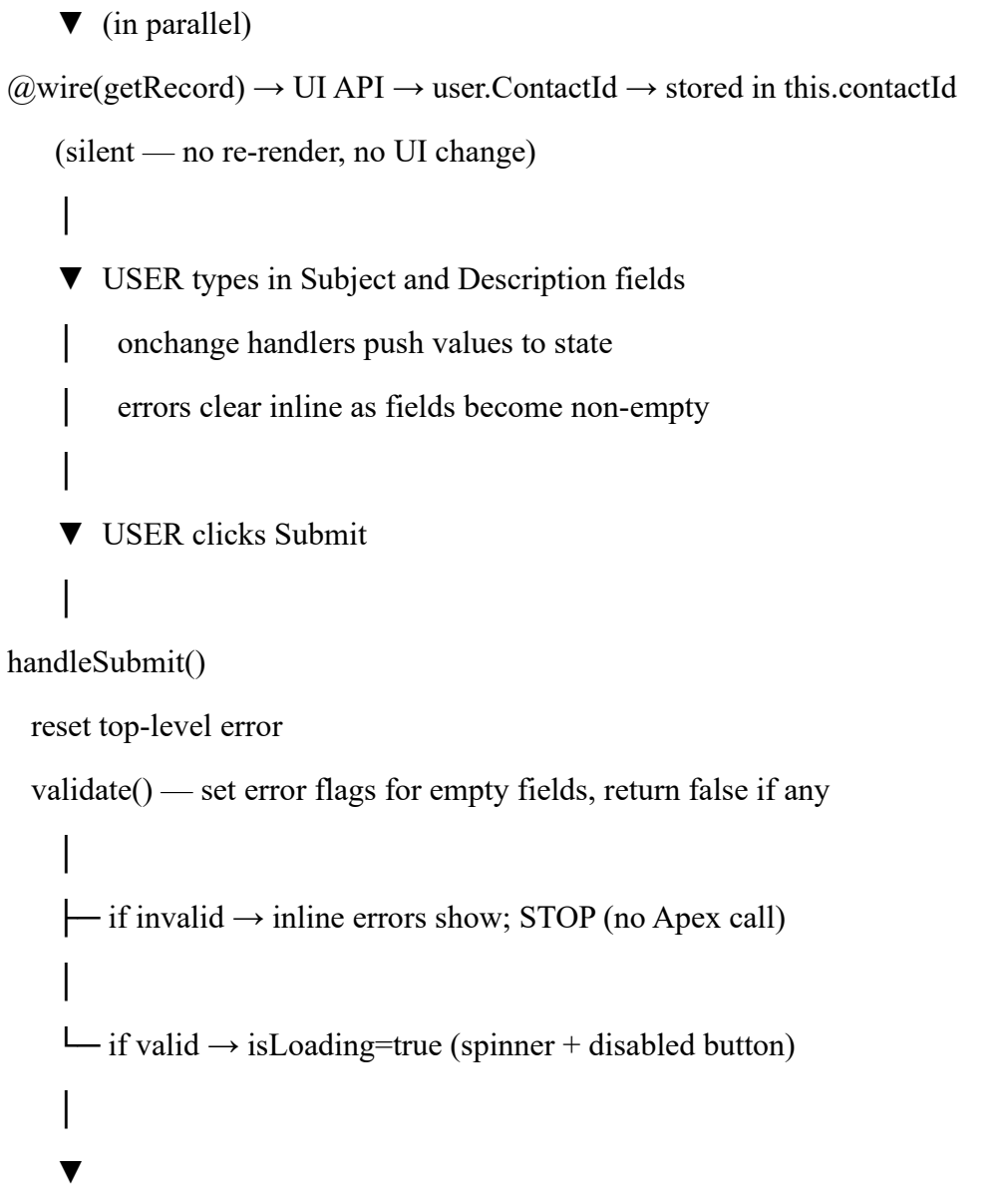
standard__namedPage is the navigation type for Experience Cloud sites — it routes by the page's *name* in the site builder, not a record Id or URL. pageName: 'home' sends the user to the page named "home" in the NeelamCare community.

Notice this is different from Quick Links, which used standard__webPage for external URLs. Inside an Experience Cloud site, standard__namedPage is the right type — it stays within the site, respects the site's URL structure, and avoids a full page reload.

The user lands on the portal home page. The current component is destroyed (its disconnectedCallback would fire if it had one), and the home page's components take over.

The flow in one picture

```
Page loads
    |
    ▼
constructor — all flags false, fields empty
    |
    ▼
Template first render: form view shown
    |
```


▼ (in parallel)
@wire(getRecord) → UI API → user.ContactId → stored in this.contactId
(silent — no re-render, no UI change)
▼ USER types in Subject and Description fields
onchange handlers push values to state
errors clear inline as fields become non-empty
▼ USER clicks Submit
handleSubmit()
reset top-level error
validate() — set error flags for empty fields, return false if any
├─ if invalid → inline errors show; STOP (no Apex call)
└─ if valid → isLoading=true (spinner + disabled button)
▼
imperative createSupportCase({subject, description, contactId})
─► Apex on server
query RecordType by DeveloperName 'Support_Case' (dynamic, portable)
build Case object with form values + Status/Origin/Priority/Type
insert newCase → fires triggers, assignment rules, flows
try/catch re-throws as AuraHandledException on failure
├─► .then(success path)
isLoading=false
isSubmitted=true

```
│    │
│    ▼ re-render — form view torn down, success view appears
│    │   "Request Submitted Successfully!"
│    │   "Go to Home" button
│    │
│    ▼ USER clicks "Go to Home" → goToHome()
│        NavigationMixin → standard__namedPage 'home'
│        Experience Cloud routes within the site
│
└─► .catch(error path)
    isLoading=false
    hasError=true, errorMessage captured
    │
    ▼ re-render — red banner on top of the SAME form
        form values intact; user can fix and resubmit
```

How this stacks up against the seven siblings

The full set now covers eight components across every common LWC pattern:

Component	Direction	Server	View structure	Distinctive feature
Quick Links	Out (nav only)	0	One view	NavigationMixin
Welcome Banner	In	1 wire (LDS)	One view	Schema imports + user Id
Case Dashboard	In	1 wire (Apex)	One view	Computed-getter visualisations
Active Campaigns	In	1 imperative	One view	Server-side date formatting
Patient Immunization	In	1 imperative	One view + modal	Client-side isUpcoming derivation

Component	Direction	Server	View structure	Distinctive feature
My Patients	In	2 imperative	3 views drill-down	State-machine flags
Featured Topics	In	2 imperative	3 views drill-down	Knowledge__kav + rich text
Raise Support Case	**In + Out + Nav**	**1 wire + 1 imperative**	**2 views (form / success)**	**Form validation + DML + post-submit navigation**

A few specific things worth knowing for the demo:

Why one wire and one imperative call. The component needs *two* round trips for two reasons. The ContactId lookup is data the component reads passively — wire is the right tool, with its automatic caching and FLS handling. The case creation is an action with side effects — imperative is the right tool, because you need explicit control over when it fires (only on Submit, after validation) and how to handle errors. Different jobs, different tools.

Why with sharing on the Apex this time. Look at the controller declaration: public with sharing class SupportCaseController. That's different from most of the others which are without sharing. This is the right call for a DML write — the record is being created on behalf of the patient, so it should respect that patient's record-level access. (For inserts specifically, with sharing mostly affects whether triggered code can read related records the user can't see.)

Querying RecordType by DeveloperName is the deployable pattern. Hardcoded Ids break across orgs; DeveloperName is metadata and travels with the schema. If asked "how would you make My Patients more robust," the same trick applies.

Why the controller doesn't return the new Case Id. A small missed opportunity. Returning newCase.Id would let the success screen show "Your case has been created as #" + the CaseNumber. Worth mentioning as an enhancement.

The race between the wire and the user clicking Submit. As mentioned earlier, if the user submits before the wire returns, contactId is undefined and the Case gets created with no Contact link. The fix is one check in validate(). Honest answer for an evaluator probing edge cases.

standard__namedPage vs standard__webPage. Quick Links uses webPage because YouTube is outside Salesforce. Here, the home page is *inside* the Experience site, so namedPage is correct — it preserves the site URL structure, respects login state, and avoids a hard reload.

No connectedCallback here. Unlike all the other components, this one doesn't load any data on mount. The wire fires automatically, and that's it — no connectedCallback is needed. The

form starts empty and waits for the user to type. This is a clean illustration of when *not* to use the lifecycle hook.

Case Assignment Trigger

Here's the data flow for the Case Assignment Trigger — the most architecturally sophisticated piece in your project, and the one most likely to draw deep questions in the evaluation.

The pieces in play

This isn't a UI flow like the LWCs — it's pure server-side automation triggered by DML. The pieces are:

1. **The trigger** — a thin one-line router on Case (before insert + before update).
2. **The handler class** — CaseAssignmentHandler, holding all the logic.
3. **Three SOQL queries** — case filtering happens in memory, doctor lookup and live-load counting hit the DB.
4. **Two assignment paths** — manual (Health Worker picked a doctor) and auto (left blank).
5. **A shared batch tracker** — keeps the 200-record bulk case correct across both paths.
6. **A Comparator class** — sorts doctors by current workload.
7. **An audit trail** — appends a note to the case Description when reassignment happens.

The component you saw earlier in the project memory was called DoctorWorkloadBalancer and had bugs (RecordType.Name vs DeveloperName, assignment-rules override). This is the rewritten version. Two architectural fixes stand out: it queries doctor loads *live* from the database instead of trusting a stored field, and it runs before insert/update so the assignment lives on the same record save (no after-insert second DML needed).

Step 0 — The trigger fires

```
trigger CaseAssignmentTrigger on Case (before insert, before update) {
    CaseAssignmentHandler.handleAssignment(
        Trigger.new,
        Trigger.isUpdate ? Trigger.oldMap : null
    );
}
```

Salesforce groups inserts/updates into batches of up to 200 records and fires the trigger once per batch. So Trigger.new is a List<Case> of 1 to 200 records; on update, Trigger.oldMap carries the previous values keyed by Id. On insert, oldMap is null.

before context is deliberate. In before context you can mutate the records directly (c.Assigned_Doctor__c = doc.Id) and the new values save with the rest of the record — no second DML, no recursion, no extra governor cost. The earlier broken version was after insert and had to do an explicit update to apply changes; that pattern fights Salesforce's grain. Before-insert/update is the right context for "set a field on the triggering record."

The handler is called with Trigger.new and oldMap (or null). Everything from here is one handler invocation.

Phase 1 — Filter the records that actually need work

```
Id clinicalRTId = Schema.SObjectType.Case
    .getRecordTypeInfosByDeveloperName()
    .get('Clinical_Case')
    .getRecordTypeId();


List<Case> manualAssign = new List<Case>();
List<Case> autoAssign   = new List<Case>();


for (Case c : newCases) {
    if (c.RecordTypeId != clinicalRTId) continue;


    if (oldMap != null) {
        Case old = oldMap.get(c.Id);
        if (c.Assigned_Doctor__c == old.Assigned_Doctor__c
            && c.Condition_Category__c == old.Condition_Category__c) {
            continue;
        }
    }


    if (c.Assigned_Doctor__c != null) {
        manualAssign.add(c);
    } else {
        autoAssign.add(c);
```

```
    }
}
```

```
if (manualAssign.isEmpty() && autoAssign.isEmpty()) return;
```

Three filters happen here, in order:

Filter 1 — Right RecordType only.
getRecordTypeInfosByDeveloperName().get('Clinical_Case').getRecordTypeId() does the lookup *dynamically*, the way SupportCaseController did. This is the fix to the bug noted in your project memory — the original code matched RecordType.Name = 'Doctor_RT' (which was wrong because Name is the localised label, not the API identifier). DeveloperName is the stable, deployable name; this works in every org.

The Support Case path (the LWC) doesn't trigger doctor assignment because its RecordType is different — it skips immediately via continue.

Filter 2 — Skip if nothing relevant changed. On update, the trigger fires for *any* edit to a Case — someone changing the priority, adding a comment, anything. You don't want to reassign doctors when the description was edited. So this block compares old vs new on the two fields that actually matter (Assigned_Doctor__c and Condition_Category__c); if neither changed, skip.

This is one of the most important bulkification details. Without this filter, an admin doing a mass update of a Status field on 200 clinical cases would re-run the entire assignment algorithm — three SOQL queries, doctor sorting, the whole machine — to change nothing. With it, those 200 cases all bypass and the handler exits via the empty-list guard.

Filter 3 — Manual vs auto. Cases where the Health Worker manually picked a doctor go to manualAssign (they may need overload-checking and reassignment). Cases left blank go to autoAssign (they need a doctor picked from scratch by specialization).

The two lists exist because the two paths have different rules:

- **Manual** keeps the HW's choice if the doctor isn't overloaded; reassigns only if the doctor has hit the 5-case limit.
- **Auto** picks from the start by specialty, falling back to General.

After filtering, if neither list has anything, return immediately. No queries, no work. This is the cheap-exit pattern that makes mass operations like data loads or background flows fast.

Phase 2 — Load all doctors once

```
Id doctorRTId = Schema.SObjectType.Contact
    .getRecordTypeInfosByDeveloperName()
    .get('Doctor_RT').getRecordTypeId();
```

```
List<Contact> allDoctors = [
    SELECT Id, Specialization__c
    FROM Contact
    WHERE RecordTypeId = :doctorRTId
      AND Available__c = true
];


if (allDoctors.isEmpty()) return;
```

A single query pulls every available doctor in the org. Two fields — Id and Specialization — are all that's needed; the doctor's name, email, etc. aren't relevant for the assignment decision.

The Available__c = true filter excludes doctors on leave or otherwise unavailable. Importantly, this query runs **once for the whole batch of 200 cases**, not once per case. If you put this query inside the loop (the classic non-bulkified mistake), 200 cases × 1 query = 200 SOQL queries, blowing through the 100-query governor limit by case 101. The pattern of "fetch all the data you need once, then loop in memory" is what bulkification means in practice.

Another early-exit: if there are no available doctors at all, return. The cases stay with whatever doctor (or lack thereof) the HW set, and the system gracefully does nothing rather than crashing.

Phase 3 — Query LIVE workload, not stored counts

This is the architectural insight that distinguishes this code from a naive approach. The instinct would be to store a Current_Cases__c counter on the Doctor (Contact) and read it. But that field would have to be kept in sync by:

- Incrementing when a case is assigned.
- Decrementing when a case is closed.
- Recalculating when a case is reassigned.
- Recovering from race conditions, batch updates, manual data changes...

That's a fragile dependency. Instead:

```
Map<Id, Integer> liveCaseCountMap = new Map<Id, Integer>();


// Initialise all doctors to 0
```

```
for (Id docId : allDoctorIds) {
    liveCaseCountMap.put(docId, 0);
}

// Overlay actual live counts from the DB
for (AggregateResult ar : [
    SELECT Assigned_Doctor__c docId, COUNT(Id) cnt
    FROM Case
    WHERE Assigned_Doctor__c IN :allDoctorIds
      AND IsClosed = false
      AND RecordTypeId = :clinicalRTId
    GROUP BY Assigned_Doctor__c
]) {
    liveCaseCountMap.put(
        (Id) ar.get('docId'),
        (Integer) ar.get('cnt')
    );
}
```

This is an *aggregate SOQL* — one query that returns per-doctor open clinical case counts, grouped. The query asks Salesforce directly: "right now, how many open clinical cases does each of these doctors have?" That number is always accurate because it's read from the actual database state, not a derived field that could drift.

Notice the two-step build:

1. **Pre-fill every doctor with zero.** A doctor with no open cases wouldn't appear in the aggregate result at all (GROUP BY only returns groups that exist). So if Dr Smith has zero cases, the aggregate skips her — but the map needs an entry for her, otherwise liveCaseCountMap.containsKey(smith.Id) returns false later and the fallback : 0 kicks in. Pre-filling makes the map complete and avoids any "doctor missing from map" edge case.
2. **Overlay the real counts.** The aggregate result drops in actual counts for doctors who have cases.

The cast (Id) ar.get('docId') is needed because AggregateResult is dynamic — the columns are untyped Object until you cast.

After this phase you have a Map<Id, Integer> of doctor → current open clinical case count, accurate as of right now. This map is the source of truth for every assignment decision below.

Phase 4 — Group and sort doctors by specialty

```
Map<String, List<Contact>> doctorsBySpec = new Map<String, List<Contact>>();


for (Contact doc : allDoctors) {
    String spec = doc.Specialization__c;
    if (spec == null) continue;
    if (!doctorsBySpec.containsKey(spec)) {
        doctorsBySpec.put(spec, new List<Contact>());
    }
    doctorsBySpec.get(spec).add(doc);
}
```

Doctors are bucketed by specialty: { 'Cardiology': [Dr A, Dr B], 'Pediatrics': [Dr C], 'General': [Dr D, Dr E] }. Doctors with no specialty set are skipped — they won't be assigned anything (a deliberate choice; you can't route by specialty if there's no specialty set).

Then each bucket is sorted by current workload:

```
for (String spec : doctorsBySpec.keySet()) {
    doctorsBySpec.get(spec).sort(
        new DoctorLoadComparator(liveCaseCountMap)
    );
}
```

Sorting uses Apex's Comparator interface (newer API, simpler than the older Comparable). The inner class:

```
private class DoctorLoadComparator implements Comparator<Contact> {
    private Map<Id, Integer> loadMap;
    public DoctorLoadComparator(Map<Id, Integer> loadMap) { this.loadMap = loadMap; }

    public Integer compare(Contact a, Contact b) {
```

```
        Integer loadA = loadMap.containsKey(a.Id) ? loadMap.get(a.Id) : 0;
        Integer loadB = loadMap.containsKey(b.Id) ? loadMap.get(b.Id) : 0;
        return loadA - loadB;
    }
}
```

For each pair (a, b), return negative if a should come first, positive if b should come first. Subtraction does this neatly: if a has 2 cases and b has 5, 2 - 5 = -3 → a comes first (less loaded). Sorted ASC by load means *the first doctor in each list is always the most under-loaded*.

The pattern of "Comparator parameterised by a Map" is the standard way to sort by external data in Apex — you can't put the comparison logic on the Contact itself because the load map isn't a field on Contact; you need a comparator that knows about both the records *and* the load lookup.

After this phase, picking a doctor for any case becomes "look up the right specialty bucket, walk down the sorted list, take the first one that isn't at the cap." That's what the next phase does.

Phase 5 — The shared batch tracker

Map<Id, Integer> assignedThisBatch = new Map<Id, Integer>();

This is the cleverest bulk-handling detail in the whole class. The liveCaseCountMap reflects the database state *as of the start of the trigger*. But during a 200-case batch, the handler will assign multiple new cases to doctors, and those assignments aren't in the database yet (they're pending in Trigger.new).

Imagine three new cases come in, all needing a Cardiologist. Without tracking:

- Case 1: Dr A has 2 cases in DB → assign to Dr A.
- Case 2: Dr A still has 2 in DB (the trigger hasn't committed yet) → assign to Dr A.
- Case 3: Dr A still has 2 in DB → assign to Dr A.

Result: Dr A ends up with 5 cases assigned in one transaction, and the bulkification illusion is broken. The "max 5 cases" rule is silently violated within a single batch.

assignedThisBatch tracks how many cases the handler has assigned to each doctor *during this run*. The effective load is computed as:

liveLoad + batchExtra

So now:

- Case 1: Dr A has 2 in DB + 0 assigned in batch = 2 → under cap → assign + bump tracker to 1.

- Case 2: Dr A has 2 in DB + 1 assigned in batch = 3 → under cap → assign + bump tracker to 2.
- Case 3: Dr A has 2 in DB + 2 assigned in batch = 4 → under cap → assign + bump tracker to 3.
- Case 4: Dr A has 2 in DB + 3 assigned in batch = 5 → AT cap → look elsewhere.

The tracker is also *shared* between the manual and auto paths. A case that came in manual-assigned might bump the tracker for Dr A, and the next case in the auto path will see that bump. Without sharing, a batch with mixed manual/auto inputs could double-load a doctor.

This kind of in-memory accumulator is one of the patterns that separates "code that works on one record" from "code that works on 200 records in a way that respects governor limits AND business rules."

Phase 6a — Manual assignment path

```
for (Case c : cases) {
    Id assignedDoc = c.Assigned_Doctor__c;


    Integer currentLoad = liveCaseCountMap.containsKey(assignedDoc) ?
liveCaseCountMap.get(assignedDoc) : 0;
    Integer batchExtra  = assignedThisBatch.containsKey(assignedDoc) ?
assignedThisBatch.get(assignedDoc) : 0;


    // Under the limit — keep HW's manual assignment
    if ((currentLoad + batchExtra) < MAX_CASES) {
        assignedThisBatch.put(assignedDoc, batchExtra + 1);
        continue;
    }


    // Overloaded — find a replacement
    ...
}
```

For each manually-assigned case:

1. Look up that doctor's effective load (DB + batch).

2. If under the cap, bump the batch tracker and keep the manual choice. **The HW's decision is respected.**
3. If at or above the cap, the manual choice is too risky — try to reassign.

The reassignment cascade is:

```
// Try same specialization first
if (category != null && doctorsBySpec.containsKey(category)) {
   reassigned = tryAssign(c, doctorsBySpec.get(category), liveCaseCountMap,
assignedThisBatch);
}

// Fall back to General
if (!reassigned && doctorsBySpec.containsKey('General')) {
   reassigned = tryAssign(c, doctorsBySpec.get('General'), liveCaseCountMap,
assignedThisBatch);
}

if (reassigned) {
   String note = '\n[Auto-reassigned: chosen doctor has reached ' + MAX_CASES
         + ' active cases. Assigned to least-loaded available doctor.]';
   c.Description = (c.Description != null) ? c.Description + note : note;
}
```

Two-level fallback: specialty match first, General as a safety net. If even the General bucket is full, the case keeps its original (overloaded) doctor — better to push slightly over the cap than to leave the case unassigned.

The Description note is the *audit trail* — a key UX touch. When the doctor opens the case, they see at the bottom of the description: "Auto-reassigned: chosen doctor has reached 5 active cases. Assigned to least-loaded available doctor." That tells them why a case appeared on their plate even though the HW didn't pick them, which avoids confusion and lets supervisors see the system's behaviour at a glance.

Phase 6b — Auto assignment path

```
for (Case c : cases) {
   String category = c.Condition_Category__c;
```

```
    Boolean assigned = false;

    // Attempt 1 — specialist by category
    if (category != null && doctorsBySpec.containsKey(category)) {
        assigned = tryAssign(c, doctorsBySpec.get(category), liveCaseCountMap,
assignedThisBatch);
    }

    // Attempt 2 — fall back to General
    if (!assigned && doctorsBySpec.containsKey('General')) {
        assigned = tryAssign(c, doctorsBySpec.get('General'), liveCaseCountMap,
assignedThisBatch);
        if (assigned) {
            String note = '\n[Auto-assigned to General doctor — all '
                + (category != null ? category : 'specialist')
                + ' doctors have reached the ' + MAX_CASES + ' case limit.]';
            c.Description = (c.Description != null) ? c.Description + note : note;
        }
    }
}
```

Same cascade — try specialist first, fall back to General — but no reassignment-from-someone-else logic, because there's no manual choice to override.

The audit note is added only when the fallback to General happens. A successful specialist assignment is silent; a forced fallback says so explicitly. This is good UX-by-data: the case description tells anyone reading it that the system had to compromise.

If both attempts fail (all specialists AND all Generals are at cap), the case stays unassigned — Assigned_Doctor__c remains null. This is a deliberate dead-end: rather than over-assigning, the system surfaces the problem (an unassigned case will be visible to whoever monitors them) so a human can decide.

Phase 7 — The tryAssign helper

```
private static Boolean tryAssign(Case c, List<Contact> candidates,
        Map<Id, Integer> liveCaseCountMap, Map<Id, Integer> assignedThisBatch) {
```

```
    for (Contact doc : candidates) {
        Integer liveLoad  = liveCaseCountMap.containsKey(doc.Id) ?
liveCaseCountMap.get(doc.Id) : 0;
        Integer batchExtra = assignedThisBatch.containsKey(doc.Id) ?
assignedThisBatch.get(doc.Id) : 0;

        if ((liveLoad + batchExtra) < MAX_CASES) {
            c.Assigned_Doctor__c = doc.Id;
            assignedThisBatch.put(doc.Id, batchExtra + 1);
            return true;
        }
    }
    return false;
}
```

The candidates list is already sorted ASC by load (Phase 4). So this walks the list from least to most loaded and takes the first doctor whose effective load is under the cap. Setting c.Assigned_Doctor__c = doc.Id in before context means the assignment saves with the rest of the record on commit. The batch tracker is bumped immediately so subsequent cases in the same batch see the updated number.

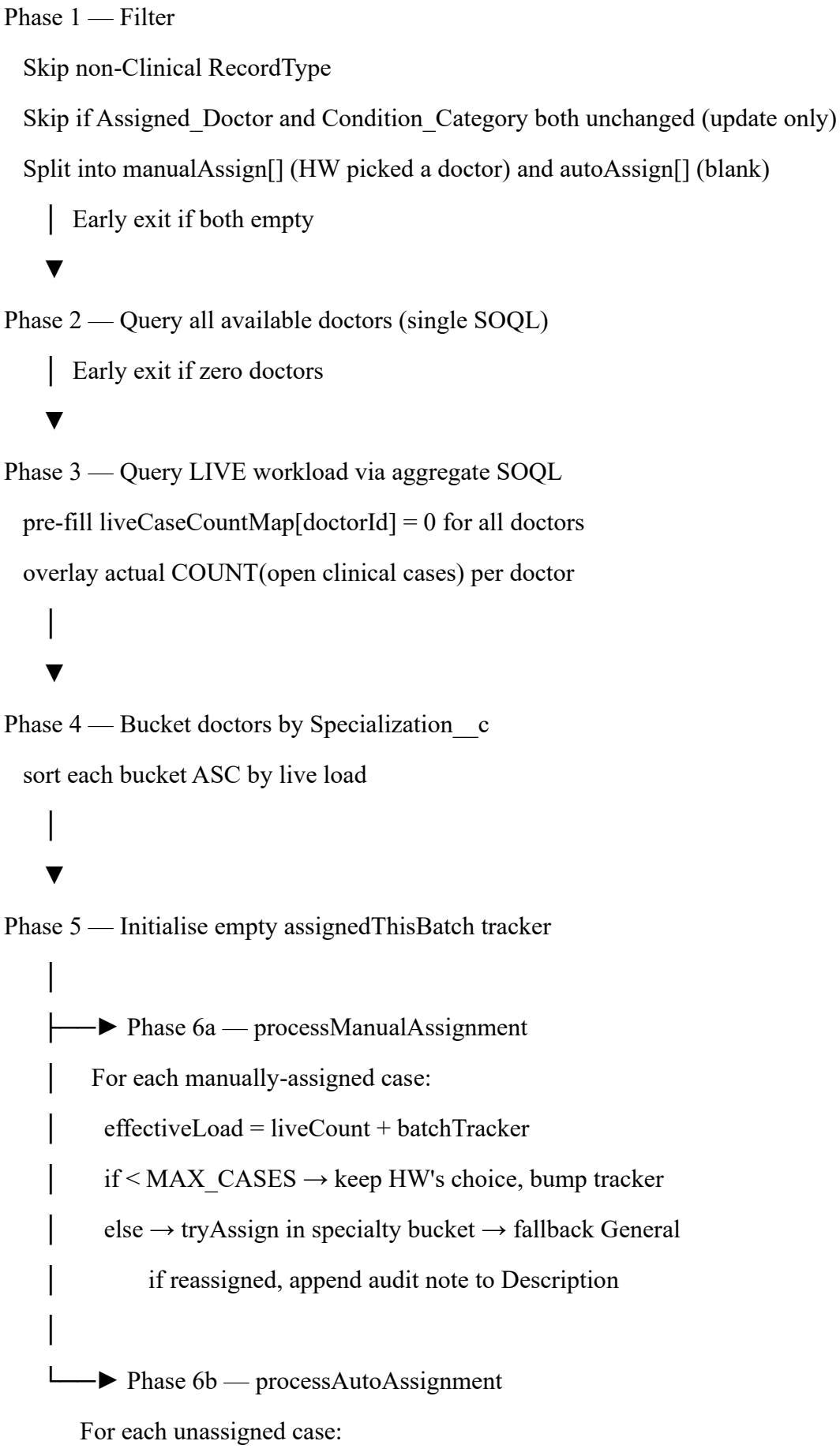
Returns true on success, false if every candidate is at cap. The caller (Phase 6a or 6b) uses this to decide whether to try another bucket or give up.

The full picture

INSERT/UPDATE of one or more Cases

CaseAssignmentTrigger fires (before insert / before update)

handleAssignment(Trigger.new, oldMap)


Phase 1 — Filter
Skip non-Clinical RecordType
Skip if Assigned_Doctor and Condition_Category both unchanged (update only)
Split into manualAssign[] (HW picked a doctor) and autoAssign[] (blank)
Early exit if both empty
Phase 2 — Query all available doctors (single SOQL)
Early exit if zero doctors
Phase 3 — Query LIVE workload via aggregate SOQL
pre-fill liveCaseCountMap[doctorId] = 0 for all doctors
overlay actual COUNT(open clinical cases) per doctor
Phase 4 — Bucket doctors by Specialization__c
sort each bucket ASC by live load
Phase 5 — Initialise empty assignedThisBatch tracker
Phase 6a — processManualAssignment
For each manually-assigned case:
effectiveLoad = liveCount + batchTracker
if < MAX_CASES → keep HW's choice, bump tracker
else → tryAssign in specialty bucket → fallback General
if reassigned, append audit note to Description
Phase 6b — processAutoAssignment
For each unassigned case:

tryAssign in specialty bucket (matched on Condition_Category)

fallback to General; append note if fallback used

if both fail, case stays unassigned

▼

Return from handler — before-context mutations save with the records

▼

Salesforce continues the order of execution:

custom validation, duplicate rules, save, after triggers,

assignment rules, workflow, flows... commit

How this stacks up against the rest

The full set of project pieces you've walked is now:

Piece	Type	Server work	Notable pattern
Quick Links	LWC	None	NavigationMixin only
Welcome Banner	LWC	Wire (LDS)	Schema imports + user Id
Case Dashboard	LWC	Wire (Apex)	Computed-getter visualisations
Active Campaigns	LWC	Imperative read	Server-side formatting
Patient Immunization	LWC	Imperative read + modal	Client-side derivation
My Patients	LWC	Two imperative reads	State machine drill-down
Featured Topics	LWC	Two imperative reads	Knowledge__kav + rich text
Raise Support Case	LWC	Wire + imperative DML	Validation + write + nav
Case Assignment Trigger	**Apex trigger**	**Before-insert/update**	**Bulk-safe load-balancing with audit**

The trigger sits at a different layer — it's not a UI flow, it's the backend automation that the **Raise Support Case** LWC and the **Health Worker app** (creating clinical cases) implicitly rely on. When a HW creates a clinical case, this trigger silently does the doctor assignment work *before* the case ever appears in any dashboard or list view.

Things worth emphasising in the demo

Before vs after. The fix from your earlier DoctorWorkloadBalancer version. Before-context mutations save with the record; there's no second DML, no recursion guard needed, and assignment rules don't fight you. This is the cleanest pattern for "set a field on the triggering record."

Dynamic RecordType lookup. Two places in this file use Schema.SObjectType.X.getRecordTypeInfosByDeveloperName().get('Y').getRecordTypeId() — once for Clinical Case, once for Doctor RT. This is the *deployable* pattern. If the evaluator asks "what would you improve in My Patients or Case Dashboard," this same lookup is the answer.

Live workload, not stored counts. Whenever the question is "should we trust a derived field?" the answer is "if you can query the source of truth cheaply, do that." A single aggregate SOQL of COUNT() with GROUP BY costs almost nothing and is always right. A Current_Cases__c field would need triggers to keep in sync and could drift.

The shared batch tracker. This is the part most candidates miss. Bulkification isn't just "don't put SOQL in a loop." It's also "make sure your in-memory accounting reflects what you've decided so far in this batch." Without assignedThisBatch, a 200-case bulk load could push a doctor far over the cap.

Audit trail in Description. "I auto-reassigned this because the chosen doctor was overloaded" is something the doctor and supervisors *need to see*. Building it into the case data itself, not a separate log, means it's visible exactly where it matters.

Two paths sharing the same primitives. Manual and Auto have different entry conditions (one starts with a doctor, one starts blank), but they share the bucket sort, the load map, the batch tracker, and tryAssign. This is the kind of structure that holds up to extension — adding a third path (say, "Emergency priority routes to whoever is online") would slot in alongside the others without rewriting anything.

Comparator with parameterised state. The Comparator pattern with a map in the constructor is the idiomatic Apex way to sort by external data. It's the kind of detail that signals "I've actually written this kind of code, not just memorised it" if it comes up.

Early exits at every gate. Phase 1 returns if both lists are empty. Phase 2 returns if no doctors. The handler tries hardest to do *nothing* when nothing needs doing. For a trigger that runs on every Case save, this matters — it's the difference between a 10ms invocation and a 200ms one when the change has nothing to do with assignment.

MAX_CASES = 5 as a constant. Instead of scattering the literal 5 through the code, it's a named constant. If the business rule changes to 7, one line moves. And the audit-trail strings reference it dynamically (MAX_CASES + ' active cases'), so the message stays in sync.

If asked "what would you still improve" — two honest answers: (1) the Available__c = true check on doctors could change between query time and assignment time if someone toggled

it concurrently, though this is mostly theoretical; (2) the Description audit trail could grow unboundedly if a case is reassigned many times in its lifetime — capping the trail or moving it to a Notes-and-Attachments child object would be cleaner long-term.

Prevent Case Close Trigger

Here's the data flow for the Prevent Case Closure trigger — the simplest of your two triggers, but a great example of the "block-save with addError" pattern that's almost impossible to do declaratively.

The pieces in play

1. **The trigger** — a one-line router on Case, before update only.
2. **The handler class** — PreventCaseClosureHandler.handleClosure, four labelled steps.
3. **A filtering loop** — narrows 200 records to only the ones actually trying to close.
4. **An aggregate SOQL** — counts prescriptions per case in one query.
5. **addError()** — blocks the save record-by-record with a friendly message.

There is no DML, no record modification, no field assignment — this trigger's only job is to **stop** something from happening. Either the save proceeds untouched, or the record is rejected with an error.

Step 0 — The trigger fires

```
trigger PreventCaseClosureTrigger on Case (before update) {
    PreventCaseClosureHandler.handleClosure(Trigger.new, Trigger.oldMap);
}
```

Two things to notice immediately:

Only before update. Inserts don't fire this. That's correct — you can't be "closing" a brand-new record because it has no previous state to compare against. A case is closed only when its Status changes *from something else* to Closed. Inserts are irrelevant to that rule.

Why before, not after. The whole point is to reject the save. In before-context, calling addError() on a record stops Salesforce from committing it and surfaces your message in the UI. In after-context, the record has already been saved; you can no longer block it cleanly. Before is the only correct context for validation-style triggers.

Phase 1 — Filter to "actually closing" cases only

```
Id clinicalRTId = Schema.SObjectType.Case
    .getRecordTypeInfosByDeveloperName()
    .get('Clinical_Case').getRecordTypeId();
```

```
List<Case> closingCases = new List<Case>();


for (Case c : newCases) {
    Case oldCase = oldMap.get(c.Id);


    Boolean isMovingToResolved =
        c.Status == 'Closed' && oldCase.Status != 'Closed';


    Boolean isClinicalCase =
        c.RecordTypeId == clinicalRTId;


    if (isMovingToResolved && isClinicalCase) {
        closingCases.add(c);
    }
}


if (closingCases.isEmpty()) return;
```

This filtering is the single most important phase. The trigger fires on *every* Case update — someone changing the description, an assignment trigger setting Assigned_Doctor, a doctor adding a comment, an admin doing a bulk Status update of cases from "New" to "In Progress." Without filtering, the rest of the handler would run on every one of those, wasting queries and CPU on cases that have nothing to do with closure.

Two conditions narrow the records:

isMovingToResolved — checks the *transition*, not just the state. The check is new = 'Closed' AND old != 'Closed', which fires exactly when the status crosses the boundary. If someone edits a *closed* case (changing its description, say), new = 'Closed' is true but old = 'Closed' is also true → not a transition → skip. If someone moves an open case to "In Progress," new = 'In Progress' → also skip.

This transition-check pattern is essential for before-update validation. Without it, a case that closed yesterday (legitimately, with a prescription) could become unsaveable today if its prescription were ever deleted — because every subsequent update would re-evaluate the rule. By checking the transition only, the rule applies *only* at the moment of closure.

isClinicalCase — Support Cases (the kind your Raise Support Case LWC creates) shouldn't require prescriptions. The dynamic RecordType lookup (same pattern as the Case Assignment trigger) excludes them.

After this filter, closingCases is the small subset of Trigger.new that needs checking. The empty-list early exit means a typical update batch — none of which touches closure — exits in microseconds without any SOQL.

Phase 2 — Collect Ids for the bulk query

```
Set<Id> closingCaseIds = new Set<Id>();

for (Case c : closingCases) {
    closingCaseIds.add(c.Id);
}
```

A trivial loop, but a critical bulkification step. The next phase queries prescriptions filtered by these Ids; collecting them upfront means one SOQL covers every closing case in the batch, regardless of how many there are.

Using a Set<Id> rather than a List<Id> is good defensive practice — even though a Case Id should never appear twice in Trigger.new, the Set guarantees uniqueness for the IN clause and is slightly faster as a lookup if you needed one later.

Phase 3 — The aggregate SOQL

```
Map<Id, Integer> prescriptionCountMap = new Map<Id, Integer>();


for (AggregateResult ar : [
    SELECT Case__c caseId, COUNT(Id) cnt
    FROM Prescription__c
    WHERE Case__c IN :closingCaseIds
    GROUP BY Case__c
]) {
    prescriptionCountMap.put(
        (Id) ar.get('caseId'),
        ((Decimal) ar.get('cnt')).intValue()
    );
}
```

This single query — one round trip to the database — counts prescriptions for every closing case at once. The result populates a map: Case Id → number of prescriptions.

A few details worth understanding:

SELECT Case__c caseId, COUNT(Id) cnt. The aliases caseId and cnt are how you name aggregate columns so you can read them out later. Without the aliases, you'd have to call them Case__c (which still works but reads worse) and expr0 (the auto-generated name for COUNT(Id), which is opaque).

GROUP BY Case__c — collapses prescriptions by parent case. So if Case A has three prescriptions and Case B has one, you get two rows back, not four.

Cases with zero prescriptions don't appear in the result. This is the most subtle point. GROUP BY only returns groups that exist; a case with no prescriptions has no rows in Prescription__c, so it has no group, so it's absent from the aggregate. That's actually fine here, because the next phase checks count == null || count == 0 — both null and zero count as "needs to be blocked."

((Decimal) ar.get('cnt')).intValue(). Aggregate results return numbers as Decimal, not Integer. The cast to Decimal first, then .intValue() to truncate to a whole number, is the standard pattern. (Same trick used in your Active Campaigns Apex for converting Target_Patients__c.intValue() to a string.)

Why this can't be done with a Flow

The comment in the code explicitly calls this out: **Flow has no aggregate SOQL support.** A record-triggered flow can do "Get Records" with filters, but it can't ask "how many child records does this record have?" in a single call. You'd have to:

1. Get all Prescriptions where Case = This_Case.Id.
2. Loop through them counting (or use the loop's collection size).
3. Repeat per case if you were processing multiple cases — which a record-triggered flow doesn't really do bulk-properly.

For one case at a time it would work, but it's much slower and breaks down badly at bulk scale (a Data Loader closing 200 cases at once). The aggregate SOQL approach is one query regardless of batch size — Flow can't match that.

A roll-up summary field on Case could store the prescription count, but that requires master-detail (which constrains the data model in other ways), and it lags slightly behind actual changes. The trigger approach is the cleanest answer.

Phase 4 — Block bad saves with addError

```
for (Case c : closingCases) {
    Integer count = prescriptionCountMap.get(c.Id);
```

```
    if (count == null || count == 0) {
        c.addError(
            'Cannot mark this case as Resolved. '
            + 'Please add at least one Prescription before closing.'
        );
    }
}
```

For each closing case, look up its prescription count and decide:

- count == null — the case wasn't in the aggregate result at all (no prescription records exist for it).
- count == 0 — defensive; shouldn't happen because empty groups don't appear, but the OR makes the check robust to any edge case.
- Otherwise the case has at least one prescription → no action, the save proceeds.

What addError does

c.addError(message) is the mechanism for blocking a single record's save in a trigger. When called:

1. The error message is attached to that record.
2. Salesforce stops the save for *that record only*.
3. In the UI, the user sees the message at the top of the record page in a red error box.
4. If they were inline-editing in a list view, that row stays in edit mode with the error message attached.
5. From an API or Data Loader, the operation returns a per-record error result the caller can inspect.

The key property: in a partial-success DML (Database.update(list, false)), one rejected case does NOT roll back the others. So if a Data Loader tried to close 200 cases and 47 of them had no prescriptions, the 153 valid ones still save, and the loader gets 47 errors with messages explaining why.

There's also c.Field__c.addError(message) to attach the error to a specific field — useful when the problem is field-level (e.g. c.Email.addError('Invalid format')). Here the error is at record level because it's not about one field but about a related-records dependency.

What does NOT happen here

It's worth noting what this trigger deliberately *doesn't* do, because each absence is a design choice:

No DML. The handler never inserts, updates, or deletes anything. Its job is purely to allow or block the incoming save. This means there's no recursion risk, no governor cost beyond the single SOQL, no risk of unintended side effects.

No field assignment. Unlike the Case Assignment trigger, which mutates c.Assigned_Doctor__c, this trigger doesn't change the case at all. The original update (with its Status = 'Closed' and whatever else the user/system changed) either goes through unchanged or gets rejected entirely.

No conditional logic on prescription content. The rule is *just* "at least one prescription must exist." It doesn't check whether the prescription is active, completed, written by the assigned doctor, or anything else. A simpler rule is easier to test, easier to explain to users, and harder to game.

No recursion guard. Because the trigger doesn't update any records, it can never re-trigger itself. The recursion-prevention pattern from your notes (static Boolean flag) is unnecessary here.

The full picture

UPDATE of one or more Cases

    │

    ▼

PreventCaseClosureTrigger fires (before update only)

    │

    ▼

handleClosure(Trigger.new, Trigger.oldMap)

    │

    ▼

Phase 1 — Filter

  Skip if Status didn't transition: old != 'Closed' AND new = 'Closed'

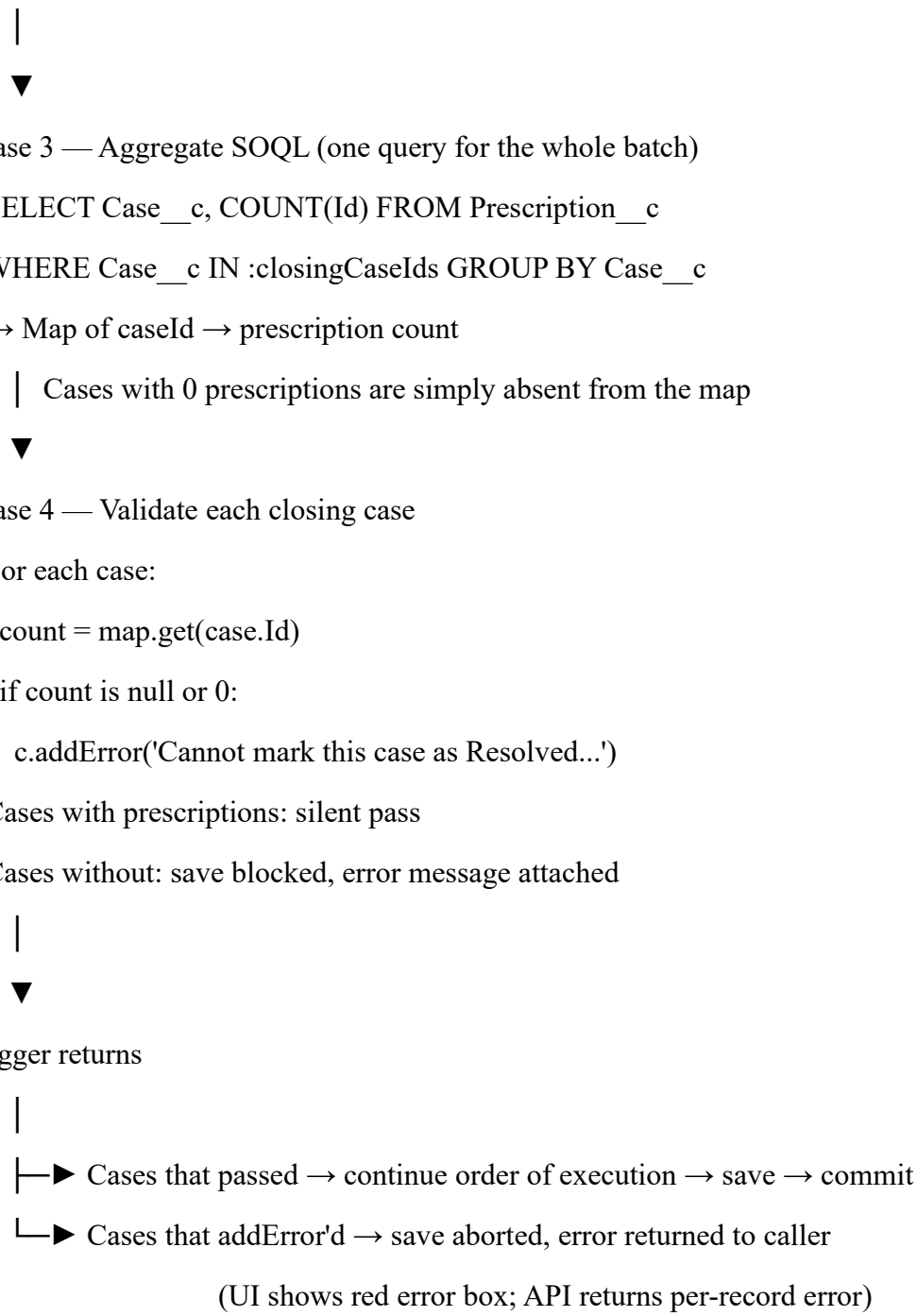
  Skip if not the Clinical Case RecordType

  Build closingCases list

    │ Early exit if empty (the common case for most updates)

    ▼

Phase 2 — Collect closing case Ids into a Set



How this fits alongside the rest

You've now seen ten pieces of NeelamCare:

Piece	Type	Role	Server work
Quick Links	LWC	Navigation only	None
Welcome Banner	LWC	Display name	Wire (LDS)
Case Dashboard	LWC	Display counts	Wire (Apex)

Piece	Type	Role	Server work
Active Campaigns	LWC	Display list	Imperative read
Patient Immunization	LWC	List + modal	Imperative read
My Patients	LWC	Drill-down list	Two imperative reads
Featured Topics	LWC	Knowledge browser	Two imperative reads
Raise Support Case	LWC	Form + DML	Wire + imperative write
Case Assignment Trigger	Apex trigger	Set field on save	Aggregate SOQL + sort + assign
Prevent Case Closure Trigger	**Apex trigger**	**Block invalid save**	**Single aggregate SOQL**

These last two triggers represent two of the three things triggers do in practice. **Case Assignment** *modifies* the triggering records (sets Assigned_Doctor__c). **Prevent Case Closure** *validates* them (blocks the save). The third thing — creating *related* records, like an after-insert that creates child Tasks — isn't in this set. Together with Salesforce's declarative tools (validation rules, flows), this covers the full automation surface.

A few things worth knowing for the demo

Why a trigger instead of a validation rule. This is the most likely evaluator question. A validation rule's formula language can't query other objects — you cannot write "block if zero child records exist" as a single formula. You'd need a roll-up summary field on Case storing the prescription count, but that requires master-detail (a one-way commitment in the data model), and roll-ups have their own gotchas. The trigger is more flexible: the rule can change ("at least one *active* prescription" or "at least one prescription written by the assigned doctor") with a simple SOQL tweak, without altering the schema.

Why before update, not after update. addError only works to block a save when called in before-context. In after-context, the record has already been written; calling addError there raises an exception rather than rejecting one record. Before is the *only* correct choice for validation triggers.

Aggregate SOQL is the right tool. A naive implementation would loop through closing cases and run one SOQL per case ("does this case have any prescriptions?"). That's 200 queries for a 200-case batch — instant governor violation. The aggregate with GROUP BY does it in one. This is exactly the pattern that the Case Assignment trigger uses for live workload counting.

The transition check new == 'Closed' && old != 'Closed' is the heart of correctness. Without it, every edit to an already-closed case would re-run the validation, which means any

closed case from before this trigger was deployed could become unsaveable if its prescriptions were later deleted. The transition pattern scopes the rule to the moment it should apply.

without sharing is the safe choice here. The handler queries prescriptions to determine count. If the running user can't see all prescriptions (community user, restricted profile), with sharing would give them an *undercount*, which could block a legitimate close. without sharing queries the truth and bases the decision on that. The security trade-off is fine because the trigger only *reads* prescriptions; it doesn't expose them.

The error message is user-facing and friendly. "Cannot mark this case as Resolved. Please add at least one Prescription before closing." — clear about what went wrong AND what to do next. Compare against a vague "Validation failed" — that's the difference between users helping themselves and users opening support cases.

What if a doctor closes a case AND adds the prescription in the same transaction? This is a worthwhile edge case to think about. If a Screen Flow updates the Case to Closed *and* inserts a Prescription in one transaction, this trigger fires during the Case before-update step, queries Prescription__c, and the new Prescription isn't visible yet (because it hasn't been inserted). The validation would fire incorrectly. Defending against this is tricky — you'd need to also check Trigger.new of an in-memory prescription, but the trigger only sees Cases. Honest answer if asked: "The current design assumes prescriptions exist before the close. A flow or LWC must order operations as: create prescription first, then close case. The Raise-and-resolve flow in our app does exactly that."

No recursion guard, no recursion risk. Since this trigger doesn't update anything, it can't re-trigger itself. The recursion-prevention static Boolean from your notes is for triggers that perform DML on their own object — this isn't one of them.

NeelamCare — Complete Evaluator Q&A Bank

Section 1 — Project Overview

Q. Introduce your project in 60 seconds.

NeelamCare is a Salesforce-native community healthcare platform built on Experience Cloud. It supports organised health camps in rural and semi-urban areas. The platform serves four user types — Admin, Health Worker, Doctor, and Patient. The Admin sets up regions, accounts, and outreach programs. Health Workers register patients at the camp, record immunizations, and open clinical cases. Doctors log into the portal, review assigned cases, diagnose, prescribe, and close the case. Patients log in and view their own immunization history, medical records, and can raise support requests. Background automation handles reminders and workload balancing. Built with 8 objects, 13 LWC components, 10 Apex classes, 4 triggers, and 8 flows.

Q. What problem does NeelamCare solve?

Rural patients don't have easy access to hospitals. Health workers visit them at community camps but traditionally worked with paper records. Doctors had no visibility into community health data. Patients lost their medical history. NeelamCare puts everything in one Salesforce org — health workers record data digitally at the camp, doctors see cases assigned to them instantly, patients access their records from the portal. It removes the paper gap and gives every stakeholder a real-time view of the patient's health journey.

Q. Why Salesforce for a healthcare project?

Salesforce's declarative tools (Experience Cloud, Flows, Sharing Rules) let you build a multi-portal, multi-user system fast without managing infrastructure. The data model is flexible — standard objects like Contact and Case map naturally to patients and clinical cases. Experience Cloud gave us a public-facing portal without building a separate web app. Apex and LWC handled the custom logic and UI that declarative tools couldn't. For a hackathon, it was the right balance of speed and capability.

Q. What are the four user types and how do they differ?

Admin is an internal Salesforce user with the HC Admin profile — sets up Regions, Accounts, and Outreach Programs. Health Worker is internal with the HC Health Worker profile — registers patients, records immunizations, and creates clinical cases. Doctor is a portal user with the NC Doctor Portal profile — linked to a Contact with the Doctor record type, logs into Experience Cloud to view and treat assigned cases. Patient is a portal user with

the NC Patient Portal profile — linked to a Contact with the Patient record type, logs in to view their own records and raise support requests.

Section 2 — Data Model

Q. Walk me through your 8 objects.

Contact is used for both Patients and Doctors via record types. Account holds the hospitals and PHCs that organise camps. Case is used for both clinical cases and support requests via record types. Region is a custom object that acts as the geographic backbone — driving sharing and routing. Outreach Program is a custom object representing a health camp event with a date, region, and organising account. Program Patient is a custom junction object linking a patient Contact to an Outreach Program — it creates the many-to-many relationship. Immunization Record is a custom object holding vaccine details linked to a patient. Prescription is a custom object in a master-detail relationship with Case — a prescription cannot exist without its case.

Q. Why did you use Contact for both Patients and Doctors instead of separate objects?

Contact is Salesforce's standard person object, and Experience Cloud portal users are inherently linked to Contacts via the ContactId field on User. If I had used a custom object for patients, I would have had to write extra Apex to bridge the portal user to their records, and standard features like Web-to-Lead conversion and Case.ContactId would not work directly. Using one object with two record types — Patient and Doctor — kept the data model clean, leveraged platform standards, and simplified the security model.

Q. Why is Program Patient a master-detail junction and not just a lookup?

A patient can attend multiple programs, and a program has many patients — that's a many-to-many relationship, which requires a junction. The first master-detail (to Program) controls the junction's ownership and sharing. More importantly, if a Program is deleted, the corresponding Program Patient records are cascade-deleted automatically — this is the right behaviour because an enrollment record without either parent is meaningless. A lookup would leave orphan records.

Q. Why is Prescription a master-detail to Case, not a lookup?

A prescription only makes sense in the context of a clinical case. If the case is deleted, its prescriptions should also be deleted. The master-detail enforces this cascade. It also allows a roll-up summary on Case (like COUNT of prescriptions) if needed, and it inherits the case's ownership and sharing — so any user who can see the case can also see its prescriptions without extra sharing rules.

Q. What is the Region object and why did you create it?

Region is a custom object representing a geographic area — Chennai North, South, West, etc. It acts as the backbone for sharing and routing. Outreach Programs and Patient Contacts both have a lookup to Region. Sharing rules use the Region field as criteria — for example, Health Workers get read/write access to Program Patients where the Region matches their assigned area. Without Region as a proper object, sharing rules couldn't be region-based, and you'd have to use picklists which can't be used in criteria-based sharing rules.

Q. How many record types do you have and on which objects?

Four record types across two objects. On Contact: Patient RT (for patients registered through the platform) and Doctor RT (for doctors who are also portal users). On Case: Clinical Case RT (for medical cases created by health workers) and Support Case RT (for support requests created by patients through the LWC form). Each record type drives different page layouts, picklist values, and automation behaviour — for example, the Case Assignment trigger only fires on Clinical Case, and the Prevent Closure trigger only fires on Clinical Case.

Section 3 — Security Model

Q. Explain your sharing model.

OWD for most objects is Private. We then open access with sharing rules. Health Workers get Read/Write on Program Patient where Is_Active is true. Doctors (as a public group) get Read/Write on Clinical Cases where Type equals Clinical. Health Workers also get Read/Write on Immunization Records where Is_Active is true. Doctors get Read Only on Contacts where Consent is true. Admins get Read/Write on Contacts where Record Type is Patient. This means each persona sees exactly the data they need — no more.

Q. Why OWD Private? Isn't that too restrictive?

Private is the safest starting point for healthcare data. Patient records, immunizations, and clinical cases are sensitive. We open access deliberately through sharing rules for each persona's legitimate need. Starting Public would mean we'd have to lock things down case by case, and it's easy to miss something. Private + share up is the Salesforce best practice for regulated data.

Q. How do portal (external) users access data?

Portal users use special Experience Cloud licenses, not standard internal licenses. They authenticate via their User record, which is linked to a Contact via ContactId. They can see

records based on OWD and sharing sets — Salesforce doesn't apply the internal role hierarchy to external users. Patients see only their own Contact and related records because OWD is Private and no sharing rule grants them access to other patients' data. Doctors see cases assigned to them because of the Public Group sharing rule on Case.

Q. What is a Public Group and how did you use it for Doctors?

A Public Group is a named collection of users used in sharing rules and folder access. It can't own records. I created the HC Doctors public group containing all users with the Doctor portal profile. A criteria-based sharing rule on Case shares any Case where Type equals Clinical with that public group at Read/Write. This means every doctor can see every clinical case — which is correct, because a doctor might need to cover for a colleague.

Q. What is a Sharing Set and did you use one?

A Sharing Set grants portal (external) users access to records related to their Contact or Account. For example, a Patient portal user automatically gets access to their own Immunization Records and Cases because those records have a Patient__c or ContactId field pointing to their Contact. Without a sharing set, the patient would be locked out of their own records. Sharing sets are the primary mechanism for Experience Cloud data access.

Section 4 — Flows & Automation

Q. What flows did you build?

NC Patient Onboarding Flow — a record-triggered flow on Contact (Patient) that fires on create. It sends a welcome email, sets the Next Checkup date to today plus three days, and creates a task for the health worker. NC Case Resolution Flow — record-triggered on Case where Status changes to Resolved. It stamps the resolved date, emails the patient, and creates a task for the health worker. Immunization Reminder Flow — a scheduled flow that runs daily at 8 AM, finds vaccines due within seven days, emails the patient, and creates a task for the health worker. It has an "Has Email" decision inside the loop to prevent crashes when a patient has no email. Raise Case — a screen flow for the patient-facing support case form.

Q. Why did you use a Flow for the onboarding rather than a trigger?

The onboarding logic — send email, set a date field, create a task — is exactly what flows are designed for. No complex conditional logic, no aggregates, no callouts. Flow handles it declaratively with no code to maintain or test. I use triggers only when flows hit their limits: aggregate SOQL, bulk processing beyond flow's capabilities, or complex data transformations. Using a trigger here would be over-engineering.

Q. Why did the Immunization Reminder Flow need an "Has Email" decision?

Flows crash on an unhandled fault. If a patient Contact has no email field populated and the flow tries to send them an email using the Send Email action, it throws an error that can propagate and fail the entire scheduled run. By adding a Decision element before the email action — checking whether the Email field is not blank — the flow gracefully skips patients with no email rather than crashing. Without it, one patient with a missing email would prevent reminders from being sent to everyone else in that batch.

Q. What is the difference between a before-save and after-save record-triggered flow?

Before-save (fast field update) runs before the record is committed. You can update fields on the triggering record with no extra DML — cheapest option. Cannot create related records or call actions. After-save runs once the record is saved with its Id. Can create/update related records, send emails, call sub-flows and Apex. The Patient Onboarding Flow uses after-save because it creates a Task (a related record) and sends an email — both need the record's Id.

Section 5 — Triggers Deep Dive

Q. How many triggers do you have and on which objects?

Four triggers. PreventDuplicateEnrollment on Program_Patient__c — before insert. PatientRiskScoreEngine on Immunization__c — after insert, update, and delete. CaseAssignmentTrigger on Case — before insert and before update. PreventCaseClosureTrigger on Case — before update only.

Q. Walk me through the PreventDuplicateEnrollment trigger.

It's a before-insert trigger on Program Patient. Its purpose is to stop a Health Worker from enrolling the same patient in the same Outreach Program twice. In the handler, I collect all the Program/Patient pairs from the incoming records. Then a single SOQL queries existing Program Patient records for those same combinations. Any new record that matches an existing one gets a addError call blocking the save with a friendly message. This is done in before-insert so no duplicate record is written to the database.

Q. Why not use a validation rule or duplicate rule instead?

A validation rule can check fields on the same record but cannot query related records for a combination match. A standard Duplicate Rule works on the same object's fields like Name or Email — it doesn't support "this pair of lookup fields already exists in another record"

logic. The trigger's combination check across two foreign keys is exactly the kind of logic that requires Apex.

Q. Explain the PatientRiskScoreEngine trigger.

It fires after insert, update, and delete on Immunization Record. When a patient's immunization history changes — a vaccine added, updated, or removed — the trigger recalculates a risk score on the related patient Contact. The risk score logic looks at completeness of vaccination, overdue doses, and frequency of updates. After-context is correct here because we're updating a related record (Contact.Risk_Score__c), not the triggering record, and we need the Immunization's Id to exist. It fires on delete as well because removing a vaccine could change the risk level.

Q. Explain the CaseAssignmentTrigger and what bugs you fixed.

It fires before insert and update on Case. Its job is to assign clinical cases to doctors by workload. The original version had two bugs. First: I queried doctors using RecordType.Name = 'Doctor_RT' — but Name is the label, which can vary; the correct filter is RecordType.DeveloperName = 'Doctor_RT', which is the stable API identifier. This bug caused the query to return zero doctors and the trigger to silently exit. Second: I initially wrote it as after-insert and did an explicit DML update to set the assignment — but Salesforce Assignment Rules run after after-triggers and overrode my field change. The fix was to refactor to before-insert/update so the assignment writes with the original save and Assignment Rules don't conflict.

Q. How does the Case Assignment logic actually work?

The handler queries all available doctors once for the whole batch. Then it queries live open clinical case counts per doctor using a single aggregate SOQL grouped by doctor — this gives accurate real-time workloads without trusting a stored counter field. Doctors are bucketed by specialisation and sorted ascending by load using a Comparator class. A shared batch tracker accumulates assignments made during this invocation so later cases in the same 200-record batch see updated counts. For manually assigned cases, the handler checks if the chosen doctor is overloaded; if so, it reassigns to the next available specialist. For unassigned cases, it picks the least-loaded specialist or falls back to General. When a reassignment happens, a note is appended to the case Description as an audit trail.

Q. Why did you use an aggregate SOQL for workload instead of storing a case count on the doctor's Contact record?

A stored counter field would need to be kept in sync: increment on assignment, decrement on close or reassignment, recalculate after bulk updates. That creates multiple additional triggers and the possibility of the count drifting out of sync due to manual data changes or governor limit partial failures. The aggregate SOQL asks Salesforce directly: "how many open clinical cases does each of these doctors have right now?" That's always the true number, queried in one call for the whole batch. More reliable, no maintenance burden.

Q. Explain the PreventCaseClosureTrigger.

It fires before update on Case. When a Clinical Case tries to transition to Closed — specifically, when the Status changes from something other than Closed to Closed — the handler queries whether at least one Prescription exists for that case using an aggregate SOQL. If the count is zero or null, it calls addError on the case, blocking the save with the message "Cannot mark this case as Resolved. Please add at least one Prescription before closing." This enforces the business rule that a doctor must prescribe before discharging a patient.

Q. Why use a trigger for this instead of a validation rule?

A validation rule formula cannot query a related object. To check "does this case have at least one prescription?" you need SOQL, which is only available in Apex. Alternatively, you could use a roll-up summary field (COUNT of Prescriptions) and reference that in a validation rule — but roll-up summaries require master-detail, which we already have here. The trigger approach is more flexible: the rule can evolve ("at least one prescription written by the assigned doctor") without schema changes, just a SOQL tweak.

Q. Why is the Prevent Closure trigger before update and not before insert?

A case can only be "closed" by changing its Status from an open value to Closed — that's an *update*, not a create. You can't close a brand-new case on insert. Also, the transition check (new.Status == 'Closed' && old.Status != 'Closed') requires access to the old values, which only exists in update context via Trigger.oldMap. Insert doesn't have old values.

Q. What is bulkification and how did you apply it in your triggers?

Bulkification means writing code that handles up to 200 records — Salesforce's trigger batch size — without hitting governor limits. The rule is: never put SOQL or DML inside a loop. In every handler I collect record Ids into a Set first, run one SOQL outside the loop, build a Map of results keyed by Id, then loop through records looking up from the Map. For the Case Assignment trigger I also maintain the shared batch tracker map so the assignment logic is accurate across all 200 records without re-querying.

Q. How do you prevent trigger recursion?

With a static Boolean flag in a helper class. Static variables live for the entire transaction. On the first invocation, the flag is false — logic runs, flag is set to true. If the trigger fires again in the same transaction (because the logic did DML on the same object), the guard check sees the flag is true and returns immediately. I applied this specifically in the PatientRiskScoreEngine trigger because updating the Contact's risk score field could theoretically re-trigger Contact-related automation.

Q. Why one trigger per object?

Salesforce doesn't guarantee execution order among multiple triggers on the same object. If I wrote a separate trigger for assignment and another for closure prevention on Case, I can't control which fires first. One trigger per object, delegating to a handler, gives a single predictable entry point. The handler can sequence logic explicitly, and if I need to add another Case trigger behaviour in future, I just add a method to the handler.

Section 6 — LWC Deep Dive

Q. Walk me through your LWC components.

Thirteen components across four personas. For the Doctor: Case Overview Dashboard (aggregate counts and progress bars), Appointed Cases (list of assigned cases with filters), My Patients (drill-down from patient list to case list to case modal), and the Welcome Banner. For the Patient: Patient Immunization (table with detail modal), Medical History (cases with prescription sub-detail), Raise Support Case (form that creates a case), and Quick Links (navigation cards). Shared public-facing: Featured Topics (Knowledge Base browser — three-view drill-down), Active Campaigns (health camp cards), the patient welcome banner, and a couple of supporting components. All share the Botanical Garden teal theme.

Q. What is the difference between @wire and imperative Apex calls and when did you use each?

@wire runs automatically when the component renders and re-runs whenever a reactive parameter (prefixed with $) changes. It's best for read-only display. I used it in the Welcome Banner to fetch the user's name via getRecord, and in the Case Dashboard to fetch aggregate case counts — both are read-only displays that don't need manual trigger control. Imperative is a Promise-based call you fire explicitly — on user action, in connectedCallback, or conditionally. I used it everywhere the fetch is triggered by a user click (My Patients drill-down, Featured Topics article list) or on component load where sequencing and explicit error

handling matters (Active Campaigns, Patient Immunization, Raise Support Case write). The key rule: wire for "always show this data," imperative for "load this when I ask."

Q. Explain the data flow in My Patients.

On load, connectedCallback fires getMyPatients() imperatively. The Apex queries Cases assigned to the current doctor to get a Set of patient ContactIds, then queries those Contacts with a subquery of their case Priorities. The JS transforms each result: computing the highest-priority badge and case count per patient. The table renders. On click, handlePatientClick reads data-id from the clicked element, calls getPatientCases imperatively with that ContactId, transforms each case adding a formatted date and flattened ownerAlias, and the cases table renders. Clicking a case row reads from the already-loaded this.cases array — no third Apex call. The modal opens with the pre-transformed data.

Q. Why does the Medical History component guard against { records: [...] } wrapping?

When you use @wire to call an Apex method that returns a subquery, the framework wraps child records in { records: [...] }. When you call the same method *imperatively*, the subquery results come back as a plain array directly. The component originally expected the wired shape and broke when called imperatively. The fix was Array.isArray(c.Prescriptions__r) ? c.Prescriptions__r : [] — check the actual shape at runtime rather than assuming one or the other.

Q. What is Lightning Message Service and where did you use it?

LMS is a Salesforce publish-subscribe mechanism for components that aren't parent-child — they can even be on different regions of the page or in Aura/VF alongside LWC. You create a Message Channel XML file defining the payload fields. A publisher component calls publish(messageContext, channel, payload). Any subscriber on the same page calls subscribe in connectedCallback and unsubscribe in disconnectedCallback. In NeelamCare, I used LMS to communicate between the filter component and the case list component on the Appointed Cases page — they're siblings with no parent-child relationship.

Q. What is the Lightning Data Service and where did you use it?

LDS lets you read and write single records without writing Apex — it handles caching, FLS, and sharing automatically. I used it in the Welcome Banner to fetch the current user's name via @wire(getRecord, { recordId: userId, fields: FIELDS }). No Apex class needed, no test class to maintain, and the record stays in sync with any other component on the page reading the same user. I also used lightning-record-edit-form (an LDS base component) in some form patterns.

Q. What is NavigationMixin and where did you use it?

NavigationMixin is a mixin you apply to LightningElement that adds a Navigate method. You call it with a PageReference object describing the destination — the type says what kind of destination, the attributes give the specifics. I used it in Quick Links to open external URLs (standard__webPage), and in Raise Support Case to navigate the patient home after successful submission (standard__namedPage with pageName: 'home', which routes within the Experience Cloud site). The key distinction is standard__webPage for external destinations and standard__namedPage for named pages within the community.

Q. How did you handle form validation in the Raise Support Case component?

Client-side validation runs before the Apex call. A validate() method checks that Subject and Description are non-blank after trimming. Critically, it checks *both* fields in every call before returning — so the user sees all errors at once rather than one at a time. Each field has a parallel error boolean (subjectError, descriptionError) that controls an inline error message and a CSS class on the form element. When the user fixes a field, onchange clears that field's error immediately — "live correction" UX. The submit button is disabled during isLoading to prevent double-clicks creating duplicate cases.

Q. Why did you use data- attributes on table rows instead of binding a handler directly to each record?

The same handler (handlePatientClick, handleRecordClick) is reused for every row. You can't create a separate handler per row in LWC. The data-id and data-name attributes on each row's clickable element carry the record identity. In the handler, event.currentTarget.dataset.id reads back the Id of the specific row that was clicked. This is the standard pattern for "which row was clicked" in an LWC table.

Q. What is the slds-backdrop slds-backdrop_open div in your modals?

It's the dimmed full-viewport overlay that appears behind the modal and in front of all page content. slds-backdrop positions a fixed, full-screen, semi-transparent dark layer. slds-backdrop_open makes it visible. It's always a sibling of the modal section, both gated by the same conditional template. Its purpose is to visually indicate that the modal is "blocking" — the user must deal with the modal before returning to the page. Without it, the modal would float over the page without any visual separation.

Q. How is isUpcoming computed in the Patient Immunization component?

In the loadRecords .then(), for each record I do: const nextDue = r.Next_Due_Date__c ? new Date(r.Next_Due_Date__c) : null and const isUpcoming = nextDue && nextDue > new Date(). If next due is in the future, isUpcoming is true. This is spread onto the record object. Inside the modal, <template if:true={selectedRecord.isUpcoming}> shows an amber warning banner. Computing this client-side means it reflects "right now" accurately — Apex-side computation would freeze the value at query time.

Q. What is lightning-formatted-rich-text and why did you use it in Featured Topics?

Knowledge article bodies (Body__c) are rich text — HTML with tags like <p>, , <ul>. If you bind raw HTML to {article.body} in a template, LWC escapes it and the user sees literal HTML tags as text. lightning-formatted-rich-text renders it as formatted content while sanitising it — stripping scripts, event handlers, and iframes. This is the safe and correct pattern for rendering user-authored HTML in LWC.

Q. Why did you use Knowledge__kav in Featured Topics, not KnowledgeArticle?

Knowledge__kav is the Knowledge Article Version object — it holds the actual content per language and per version. Querying it gives you the published text. KnowledgeArticle is the master object that groups versions by KnowledgeArticleId. For display, you always query Knowledge__kav filtered by PublishStatus = 'Online', IsLatestVersion = true, and Language = 'en_US' to get exactly the current published English content. KnowledgeArticleId is then the stable Id you pass back to the detail query — it doesn't change between versions.

Section 7 — Apex Design Decisions

Q. What is a handler-service pattern and why did you use it?

The trigger itself contains only routing logic — it reads Trigger.operationType and calls the appropriate handler method. No business logic lives in the trigger file. The handler class holds all the real logic. This keeps logic testable (you can call the handler directly in a test without creating trigger context), prevents duplication when multiple triggers on the same object need shared code, and makes it easy to add new behaviour by adding a method rather than writing another trigger.

Q. What is with sharing vs without sharing and how did you decide?

with sharing enforces the running user's record-level sharing — they can only query/DML records they can see. without sharing ignores sharing (system mode for records). I used without sharing for controllers that need org-wide counts or data the running user might not own — like CaseDashboardController (counting all clinical cases, not just the doctor's), ImmunizationController (accessing a portal user's own records which might be below their

sharing threshold), and ActiveCampaignsController (showing public campaign data to all users). I used with sharing for SupportCaseController because it's creating a record on behalf of the user — it should respect their context. inherited sharing on reusable classes would have been the safest choice for shared utility code.

Q. Why did you use Map<String,String> as return types in some Apex methods?

In FeaturedTopicsController and ActiveCampaignsController, returning a Map<String,String> instead of the raw sObject serves three purposes. First, it lets me rename fields — KnowledgeArticleId becomes id, Program_Type__c becomes campType. Second, it coerces types — Dates get formatted to locale strings, Decimals get converted to integer strings. Third, it null-coalesces — every field always has at least an empty string, so the LWC never gets undefined. The trade-off is tighter coupling between Apex and LWC, and the LWC can't benefit from field-level metadata.

Q. What is @AuraEnabled(cacheable=true) and when should you NOT use it?

cacheable=true tells the Lightning Data Service to cache the method's result on the client, improving performance. It's required for methods called with @wire. The constraint is the method must be read-only — no DML, no database writes of any kind. Using cacheable=true on a method that performs DML would corrupt the LDS cache by mixing stale reads with fresh writes. createSupportCase and any other write method must omit it.

Q. What is the AuraHandledException and when do you throw it?

AuraHandledException is the type that LWC .catch blocks correctly receive as { body: { message: '...' } }. When a standard Apex exception propagates to the client, the error message includes stack traces and internal details — ugly and potentially exposing. Wrapping in AuraHandledException gives you control: catch the real exception, throw new AuraHandledException(e.getMessage()) with a clean user-friendly message, and the LWC .catch reads error.body.message neatly. I used it in SupportCaseController.createSupportCase.

Q. How did you query the RecordType Id dynamically?

Using Schema.SObjectType.Case.getRecordTypeInfosByDeveloperName().get('Clinical_Case').getRecordTypeId(). The DeveloperName is the stable, deployment-safe identifier for a record type — it doesn't change between orgs. Hardcoding the 18-char Id (like '012dM00000D4tODQAZ') would break when deployed to a new sandbox or production org because Ids are environment-specific. The dynamic lookup works everywhere.

Q. What is Database.Batchable and did you use it?

Database.Batchable is the interface for Batch Apex — processing large data volumes in chunks. I built a WeeklyPatientSummaryBatch class implementing it, with start returning a QueryLocator for all patients, execute processing each chunk (generating summary records or updating fields), and finish sending a completion notification. The Immunization Reminder Flow replaced the need for a batch for reminders, so the batch class handles the heavier weekly summary aggregation. It's scheduled via a Schedulable class using a CRON expression.

Section 8 — Design Decisions & Trade-offs

Q. Why did you choose Experience Cloud over a Visualforce portal?

Experience Cloud gives you a drag-and-drop builder, built-in responsive design, LWC component compatibility, and managed authentication for external users. Visualforce would require hand-coding every page, managing authentication manually, and fighting Locker Service issues. For a portal serving patients and doctors on mobile, Experience Cloud was the right choice — faster to build, better UX out of the box, and natively integrated with Salesforce's security model.

Q. When did you choose a trigger over a flow?

Three situations. First, when the logic requires aggregate SOQL — flows have no GROUP BY support. The case closure validation (count prescriptions per case) and the workload balancer (count open cases per doctor) both need aggregate SOQL, so they're triggers. Second, when the logic must be bullet-proof in bulk — Flows have documented limits on the number of records processed per transaction, and complex loops can breach those. Third, when you need to block a save with an error message — only addError in a before-trigger does this natively; flows can't abort a save in the same way.

Q. What is the race condition risk in Raise Support Case and how would you fix it?

The @wire that fetches the ContactId and the user's form-fill happen in parallel. If a very fast user submits before the wire returns, this.contactId is undefined and the case is created with no Contact link — orphaned from the patient's record. The fix is a validate() check: if (!this.contactId) { this.hasError = true; this.errorMessage = 'Still loading, please wait.'; return false; }. This is not in the current code — an honest "what would you improve" answer.

Q. What hardcoded values exist in your project and how would you improve them?

Three. The RecordType Id '012dM00000D4tODQAZ' in CaseDashboardController and HealthConnectController — fix by querying dynamically with getRecordTypeInfosByDeveloperName. The locale 'en-IN' hardcoded in several JS files — fix by either removing the locale argument (lets the browser use the user's locale) or fetching it from the User record. The doctor match Assigned_Doctor__r.Name = :currentUser.Name in My Patients and Case Dashboard — fix by matching on Id: Assigned_Doctor__c = :currentUser.ContactId. Two doctors with the same name would collide with the name-based approach.

Q. How would you make NeelamCare production-ready?

Several improvements. Replace hardcoded RecordType Ids with dynamic lookups. Add test classes for all Apex (currently missing — the project is in a dev org without 75% coverage). Add a validate() ContactId check in the support case LWC. Move the Botanical Garden teal colour tokens into a Custom Metadata type so they can be changed without code deployment. Add pagination to My Patients and Medical History for patients with many records. Add an audit object to track assignment history instead of embedding it in the Description. Add Named Credentials for any external callouts rather than hardcoding endpoints.

Q. Why did you use the Botanical Garden teal theme?

The project is a community healthcare platform — the design language should feel calm, clinical, and trustworthy, not corporate grey or startup blue. Teal sits between green (health, nature, growth) and blue (technology, trust). The specific palette was derived from the Salesforce SLDS teal family but adjusted for stronger brand identity: #0F6E56 header, #E1F5EE surface, #9FE1CB borders, #04342C text, amber #EF9F27 accent for calls-to-action. It maintains WCAG contrast ratios on the main text paths.

Q. What was the hardest bug you fixed?

The DoctorWorkloadBalancer trigger. The original version queried doctors with RecordType.Name = 'Doctor_RT' — the label, not the API name — which returned zero rows and caused silent exit. After fixing that, Assignment Rules (which run after after-triggers in the order of execution) overrode the trigger's field assignment because the trigger was after-insert and tried to do an update. The fix was moving to before-insert/update context so the field assignment writes with the original save, before Assignment Rules run. Understanding the order of execution was the key insight — without knowing that Assignment Rules run at step 7, after after-triggers at step 6, the bug would have been hard to diagnose.

Q. What did you learn from this project?

Several things. First: the order of execution matters — debugging the workload balancer trigger was impossible without understanding where Assignment Rules sit relative to before and after triggers. Second: bulkification isn't just "no SOQL in loops" — the shared batch tracker in the assignment handler taught me that in-memory state needs to reflect pending changes across the batch. Third: imperative vs wire isn't arbitrary — wire for always-on display, imperative for user-triggered actions. Fourth: Experience Cloud portal users access data through ContactId, not the standard internal sharing model — understanding that bridge unlocked the whole patient-facing security design. Fifth: test early. Not having test classes now is a gap that would block production deployment.

Q. If you had more time, what would you add?

A real-time notification when a new case is assigned to a doctor — using Platform Events and a Streaming API subscription in an LWC. A mobile-optimised PWA wrapper for health workers who work in the field with limited connectivity. A custom report type for "Contacts with Immunization Records with Upcoming Due Dates" for admin dashboards. OCR or file upload on the support case form so patients can attach photos of prescriptions or test results. Integration with an external pharmacy system via a Named Credential REST callout to auto-fill prescription drug information.